

Slovenská technická univerzita v Bratislave

Fakulta informatiky a informačných technológií

Nikola Šašinková

**Automatické generovanie prototypov na základe
návrhových modelov**

Bakalárska práca 1

Vedúci práce: Ing. Juraj Vincúr, PhD.

Obsah

Úvod.....	3
1. Stav riešenia problému v literatúre	5
2. Kategorizácia podobných riešení a ich charakteristické znaky	9
3. Identifikované nedostatky a motivačné faktory pre naše riešenie	12
4. Zdôvodnenie voľby technológií a spôsobu riešenia	15
5. Stručný opis plánovaného spôsobu riešenia	22
a. Vetva doménových objektov	22
b. Vetva používateľského rozhrania	24
6. Zhrnutie a pozícia riešenia voči existujúcemu stavu poznania	39
7. Opis implementácie - BP1	42
Postup spustenia addinu.....	53
Zhodnotenie aktuálneho stavu riešenia.....	54
Plán ďalšieho rozvoja	54
Zdroje.....	55

Úvod

V praxi vývoja softvéru sa pravidelne potvrdzuje, že jednou z najväčších príčin nezhôd medzi predstavou zákazníka a výslednou implementáciou nie je samotné programovanie, ale neskoré alebo nedostatočné overenie požiadaviek a interpretácie návrhu. Aj keď tímy využívajú formálne modely a návrhové artefakty, primárne slúžia analytikom a vývojárom. Zákazník väčšinou UML diagramom nerozumie alebo ich nedokáže spoľahlivo interpretovať v kontexte reálneho používania systému. Dôsledkom je komplikovanejšia komunikácia, vyššie riziko nesprávne pochopených požiadaviek a následne aj nákladné zmeny v neskorších fázach projektu.

Preto sa v praxi ako efektívny spôsob overovania návrhu a požiadaviek využíva prototyp používateľského rozhrania. Používateľské rozhranie zároveň predstavuje pre zákazníka najpriamočiarejšiu interpretáciu toho, ako bude systém reálne fungovať - umožňuje rýchlo diskutovať o tom, ako bude systém vyzerat', aké kroky bude používateľ vykonávať, či sú názvy prvkov zrozumiteľné a či zodpovedajú reálnemu používaniu systému v praxi. Včasná vizualizácia návrhu v podobe prototypu typicky znižuje chybovosť (odhalí nesprávne interpretácie skôr, než sa premietnu do implementácie), šetrí čas a zefektívňuje spoluprácu medzi zákazníkom, analytikom a vývojovým tímom.

Problém verifikácie požiadaviek a návrhu sa štandardne rieši viacerými prístupmi. Časté je ručné prototypovanie vo vizuálnych nástrojoch, kde sa dá rýchlo iterovať so zákazníkom, no výsledok je často oddelený od formálneho modelu a od implementácie. Ďalším prístupom je manuálna implementácia UI priamo v cieľovej technológii, prípadne s využitím vizuálnych GUI návrhárov. V oblasti modelom riadeného vývoja existujú aj postupy generovania častí systému z modelov, typicky doménových tried, kostier kódu alebo šablón vrstiev. Pri generovaní UI sa používajú rôzne transformačné reťazce alebo špecifické popisné jazyky a anotácie, ktoré dopĺňajú model o informácie potrebné pre UI.

Napriek tomu ostávajú otvorené problémy, ktoré znižujú praktickú použiteľnosť takýchto riešení. Jedným z najčastejších je rozpad konzistencie medzi modelom a kódom: Ak sa prototyp alebo implementácia upravujú manuálne, model prestáva odrážať skutočný stav systému. Ďalším problémom je, že vygenerované UI býva často len orientačné a neodráža konkrétnu vizuálnu štruktúru návrhu, čo znižuje jeho hodnotu pri komunikácii so zákazníkom. Z praktického hľadiska je náročná aj integrácia do používaného vývojového prostredia: Ak je generovanie postavené na

externých exportoch a doplnkových nástrojoch, zvyšuje sa zložitosť procesu a potreba presnej konfigurácie, čím sa znižuje šanca, že bude riešenie dlhodobo používané v reálnej praxi.

Táto práca na uvedené skutočnosti nadväzuje zameraním sa na oblasť, ktorá má najväčší priamy dopad na validáciu so zákazníkom - generovanie používateľského rozhrania vo forme prototypu. Dôraz v analýze je kladený na generovanie používateľského rozhrania: Ak zákazník nerozumie UML diagramom, práve automaticky generovaný prototyp slúži ako prepojenie medzi formálnym návrhom a praktickým používaním systému. Cieľom práce je preto preskúmať a navrhnúť postup, ktorý umožní generovať spustiteľný prototyp používateľského rozhrania priamo z návrhových modelov tak, aby sa urýchlila spätná väzba, znížila chybovosť a skrátil čas medzi návrhom a overením.

Konkrétne sa práca sústreďuje na automatizované generovanie používateľského rozhrania z modelu (vrátane zachytenia štruktúry a rozloženia prvkov podľa návrhu) a na podporu procesu, v ktorom zmeny vyplývajúce zo spätnej väzby možno realizovať na úrovni modelu a následne ich opakovane premietnuť do vygenerovaného prototypu. Tým sa podporuje konzistencia medzi modelom a kódom, minimalizuje sa priestor pre chyby vznikajúce pri manuálnom prepise a zvyšuje sa efektivita práce pri iteratívnom spresňovaní požiadaviek. Výsledkom má byť prístup, ktorý zjednoduší komunikáciu so zákazníkom, umožní skoršie overenie návrhu a prispeje k plynulejšiemu prechodu od modelu k implementácii.

1. Stav riešenia problému v literatúre

Výskum zameraný na automatizované generovanie používateľských rozhraní z modelov môžeme členiť podľa typu vstupných modelov a podľa cieľovej platformy, pre ktorú sa výsledný systém generuje. Z hľadiska vstupných modelov sme identifikovali práce vychádzajúce z UML doménových a use-case modelov, zo scenárov a kolaboračných diagramov, z BPMN modelov podnikových procesov či z UML modelov doplnených o formálne špecifikácie. Z hľadiska cieľovej platformy sa riešenia zameriavali na webové prototypy, desktopové GUI, multimodálne a multiplatformové rozhrania, prípadne mobilné aplikácie.

Článok „Automatic Generation of User Interfaces from Domain and Use Case Models“ navrhol prístup, v ktorom sa automaticky generoval prototyp používateľského rozhrania z doménového modelu a modelu prípadov použitia. Základom boli UML diagramy tried a use-case diagramy doplnené o podmienky a obmedzenia špecifikované v OCL. Cieľom bolo umožniť skorú validáciu „vykonateľného jadra“ systému prostredníctvom vygenerovaného UI a zároveň podporiť konzistenciu medzi use-case a doménovým modelom. Článok zdôraznil, že bežné UML modely bývali často neúplné a neformálne, preto nebolo možné automaticky overovať ich konzistenciu. Navrhovaný prístup preto sprísnil model pomocou OCL a využil tieto formálne špecifikácie aj na obohatenie použiteľnosti generovaného UI (napríklad odvodením validačných pravidiel z „preconditions“). [1]

Článok „Designing GUI Components from UML Use Cases“ predstavil metódu UML-GMM, ktorá konštruovala GUI komponenty z dvoch typov UML modelov: Use-case diagramov a diagramov aktivít. Výsledkom transformácie bol UML diagram tried reprezentujúci GUI komponenty, z ktorého bolo možné generovať úryvky kódu považované za GUI prototypy. Táto metóda formálne prepájala aktérov, prípady použitia a ich správanie s oknami („framami“ a „appletmi“) Java Swingu. Vzťahy generalizácie a „include“ v use-case modeli sa v navrhovanej metóde premietali do hierarchie a kompozície grafických komponentov: Špecializované prípady použitia viedli k odvodeným GUI prvkom, zatiaľ čo vzťahy „include“ zabezpečovali znovupoužitie častí rozhrania vo forme pod-komponentov. Autori sa vedome rozhodli pre Java Swing ako cieľovú implementačnú platformu, pretože bol vývojárom dobre známy a umožňoval priamo mapovať navrhované GUI komponenty na existujúce triedy knižnice Swing. [2]

Článok „Generating User Interface Prototypes from Scenarios“ sa zameriaval na generovanie UI prototypov zo scenárov reprezentovaných UML kolaboračnými diagramami. Z týchto scenárov autori automaticky odvodzovali stavové diagramy pre jednotlivé objekty a následne z formálnej špecifikácie generovali prototyp používateľského rozhrania, ktorý slúžil na validáciu požiadaviek. Kľúčovou myšlienkou článku bolo, že scenáre zachytávali konkrétne interakcie medzi aktérmi a systémom, ktoré bolo možné transformovať na presnejší model správania systému (napríklad stavové diagramy) a ten následne využiť na generovanie používateľského rozhrania aj na formálnu analýzu správania. [3]

Iný smer reprezentovali práce, ktoré integrovali GUI prototypovanie priamo do UML nástrojov formou profilov a pluginov. Článok „Integrating GUI Prototyping into UML Toolkit“ definoval UML profil pre GUI prototypovanie a popísal jeho implementáciu ako plugin do nástroja MagicDraw. Profil obsahoval stereotypy pre hlavné GUI komponenty, ktoré zodpovedali klasickým knižniciam, akú predstavuje Java Swing. Plugin zabezpečoval, že UML prvky označené týmito stereotypmi sa v nástroji zobrazovali priamo ako konkrétne grafické komponenty používateľského rozhrania, čo umožnilo navrhovať a vizuálne prechádzať postupnosť obrazoviek aplikácie a zároveň prepájať takto vzniknuté prototypy s ďalšími UML prvkami. Toto riešenie prinieslo vysokú mieru integrácie GUI pohľadu do CASE nástroja, avšak jeho hlavným cieľom bola vizuálna prototypizácia, samotné generovanie produkčného zdrojového kódu zostalo v rovine možností naznačených ako ďalší krok. [4]

V oblasti jazykov na popis používateľských rozhraní sa etabloval najmä smer UIDL („User Interface Description Language“). Článok „USIXML: a User Interface Description Language for Specifying Multimodal User Interfaces“ definoval jazyk USIXML, ktorý umožňoval špecifikovať používateľské rozhrania na viacerých úrovniach abstrakcie od abstraktného UI cez konkrétne UI až po finálnu UI implementáciu pre konkrétne zariadenia a kontexty použitia. USIXML podporoval viacero spôsobov postupu vývoja („top-down“, „bottom-up“, „middle-out“) a opieral sa o transformačný prístup medzi modelmi. [5] Nástroj FlexiXML, predstavený v článku „FlexiXML - A portable user interface rendering engine for UsiXML“ zabezpečoval zobrazenie a animáciu používateľských rozhraní popísaných v USIXML a slúžil najmä na rýchle prototypovanie a analýzu návrhov. Tieto riešenia však vychádzali zo špeciálneho jazyka a jeho vlastného nástrojového prostredia, ktoré stálo mimo bežných UML CASE nástrojov používaných v priemysle. [6]

V kontexte webových aplikácií článok „Automatic Method of Generating a Web Prototype Employing Live Interactive Widget to Validate Functional Usability Requirements“ navrhol metódu automatizovaného generovania webového prototypu, ktorý obsahoval tzv. „Live Interactive Widgets (LIW)“ - interaktívne widgety vykonávané okamžite po vzniku udalosti (napríklad validátory vstupu, kontextová pomoc a podobne). LIW boli modelované ako špecifické vzory vyjadrené v UML diagramoch, ktoré sa integrovali s jednoduchým modelom prechodu medzi obrazovkami. Výsledný prototyp slúžil na overenie funkčných požiadaviek súvisiacich s použiteľnosťou. Článok ukázal, že takto generované prototypy výrazne uľahčovali získavanie a overovanie tzv. funkčných požiadaviek na použiteľnosť, avšak riešenie bolo špecificky zamerané na webové používateľské rozhranie a na obohatenie prototypov o interaktívne podporné mechanizmy. [7]

Článok „Model-driven GUI generation and navigation for Android BIS apps“ predstavil prístup ku generovaniu GUI v prostredí mobilných aplikácií - navrhol techniku generovania Android GUI pre podnikové informačné systémy na základe anotovaných UML diagramov tried. Špecifikácia obsahovala anotácie (@StartingPoint, @list, @display, @creation a pod.), ktoré určovali počiatočné obrazovky, zoznamové a detailné pohľady a atribúty pre CRUD operácie. Navigácia v generovanom používateľskom rozhraní bola navrhnutá tak, že prechádzanie doménového modelu (cez asociácie a kardinality) sa priamo premietalo do prechodov medzi obrazovkami. Vďaka tomu bolo možné z jediného modelu vygenerovať podstatnú časť prezentačnej vrstvy Android aplikácie. [8]

Dôležitú časť tvorili prístupy, ktoré vychádzali z BPMN modelov procesov. Metóda EduBPMN, popísaná a rozšírená v článku „New Transformation Rules for the EduBPMN Method to Generate Graphical User Interfaces from BPMN“ definovala sadu transformačných pravidiel umožňujúcich generovať alternatívy dizajnu GUI z BPMN modelu doplneného o UML diagram tried. V rozšírenej verzii tejto metódy boli doplnené nové transformačné pravidlá a stereotypy, ktoré umožnili automaticky generovať návrhy formulárov, tlačidiel, hypertextových odkazov a líšt s hlavným menu z BPMN štruktúr, akými sú „exclusive gateways“, podprocesy či „lanes“. Takto vzniknuté alternatívy GUI boli reprezentované stereotypmi a bolo možné ich automaticky generovať do HTML5. [9]

Osobitnú líniu výskumu predstavujú prístupy, ktoré sa sústredili na prepojenie formálnych špecifikácií s UML diagramami a na vizualizáciu výsledkov analýz. Článok „Visualization and Interpretation of Analysis Results within the Context of Formalized UML Diagrams“ predstavil systém MINERVA, ktorý pracoval s UML modelmi, k nim generovanými formálnymi špecifikáciami v jazyku Promela a s rôznymi analýzami („model checking“, simulácia, dokazovanie). Výsledky analýz boli vizualizované priamo v kontexte pôvodných UML diagramov, čo podporovalo detekciu chýb v návrhu a postupné spresňovanie modelov. Tieto práce dokumentovali význam tesnej väzby medzi modelom, analýzou a vizualizáciou, no nezaoberali sa priamo generovaním konkrétneho GUI kódu. [10]

Tieto prístupy spájalo to, že vychádzali z princípov modelom riadeného vývoja: Zo semiformálnych alebo formálnych modelov sa generovali prototypy používateľského rozhrania - často primárne na validáciu požiadaviek, niekedy aj s ambíciou znovupoužiteľného kódu. Rozdiely však boli v tom, aké modely sa považovali za vstupné, aká bola cieľová platforma (web, desktop, mobil, multimodálne UI) a do akej miery boli nástroje integrované s bežnými CASE platformami.

2. Kategorizácia podobných riešení a ich charakteristické znaky

Na základe preštudovaných prác môžeme prístupy ku generovaniu GUI z modelov rozčleniť do niekoľkých kategórií, ktoré sú z hľadiska nášho zámeru kľúčové.

Prvou kategóriou boli prístupy založené na UML doménových a use-case modeloch. Sem patrili riešenia, ktoré vychádzali z diagramov tried doplnených o OCL podmienky a use-case modely, z ktorých generovali prototypy formulárov pre podnikové aplikácie, ako to ukázal článok „Automatic Generation of User Interfaces from Domain and Use Case Models“. [1]

Zvyčajne išlo o používateľské rozhrania založené na formulároch, ktoré umožňovali skorú validáciu doménového modelu a transakcií nad ním. Spoločným znakom týchto prístupov bolo, že pracovali s bežnými UML diagramami, ale často vyžadovali dodatočnú formalizáciu (OCL) a dopĺňujúce modely (napr. explicitný model dialógu alebo abstraktného UI).

Druhou kategóriou boli prístupy využívajúce scenáre a behaviorálne UML diagramy (kolaboračné, sekvenčné a stavové diagramy) ako primárny zdroj informácií. Článok „Generating User Interface Prototypes from Scenarios“ ukázal, že explicitné modelovanie interakcií medzi objektami v rámci prípadov použitia je možné transformovať na model správania systému (vo forme stavových automatov) a z neho následne odvodiť prototypy používateľského rozhrania, ktoré zachytávajú spôsob, akým používateľ so systémom pracuje. [3]

V porovnaní s prístupmi založenými len na statických diagramoch tried kládli tieto metódy väčší dôraz na správanie a sekvenčnosť interakcií, no používateľské rozhranie bolo primárne nástrojom na demonštráciu správania a menej často sa generoval plnohodnotný kód pripravený na rozšírenie.

Tretiu kategóriu tvorili metódy vychádzajúce z BPMN a procesných modelov. Metóda EduBPMN, ako bola predstavená v článku „New Transformation Rules for the EduBPMN Method to Generate Graphical User Interfaces from BPMN“ transformovala vybrané časti BPMN diagramov do návrhov obrazoviek a GUI komponentov, pričom jednotlivým BPMN vzorom priradzovala sady alternatív dizajnu (napr. prepínače typu „radio button“ alebo skupiny tlačidiel pre „exclusive gateways“). [9]

V tejto kategórii bolo GUI chápané najmä ako prostriedok realizácie krokov biznis procesu. Modelom riadená transformácia sa zameriavala na korešpondenciu medzi procesnými uzlami a

komponentmi formulára, často vo webovom prostredí. Pozícia jednotlivých prvkov na obrazovke tu nebývala hlavným predmetom pozornosti, skôr logická dostupnosť ovládacích prvkov pre jednotlivé kroky procesu.

Štvrtou kategóriou boli prístupy zavádzajúce vlastné jazyky na popis používateľských rozhraní (UIDL), ako je USIXML predstavený v článku „USIXML: a User Interface Description Language for Specifying Multimodal User Interfaces“. [5]

V týchto prístupoch sa rozhranie špecifikovalo v špecializovanom XML jazyku, ktorý umožňoval viacúrovňovú abstrakciu a transformácie medzi abstraktným, konkrétnym a finálnym (implementácia) rozhraním. Nástroj FlexiXML, popísaný v článku „FlexiXML - A portable user interface rendering engine for UsiXML“ potom slúžil ako interpretačné prostredie, ktoré dokázalo tieto špecifikácie interpretovať a vizuálne prezentovať používateľovi. [6]

Výhodou bola vysoká všeobecnosť a podpora rôznych kontextov použitia, nevýhodou bola nutnosť zaviesť ďalší jazyk a používať k nemu osobitné nástroje mimo bežných UML CASE nástrojov.

Piatou kategóriou boli nástroje integrujúce GUI prototypovanie do UML CASE prostredia, typicky prostredníctvom UML profilov a plugin architektúr. Článok „Integrating GUI Prototyping into UML Toolkit“ demonštroval, že bolo možné rozšíriť UML o profil so stereotypmi pre GUI prvky a cez plugin zabezpečiť, aby sa tieto stereotypované prvky zobrazovali priamo ako reálne komponenty. [4]

Tieto riešenia excelovali v prepojení pohľadu na používateľské rozhranie s ostatnými UML pohľadmi (dátový model, stavové diagramy pre navigáciu GUI) a v podpore sledovania väzieb medzi jednotlivými artefaktmi. Automatizované vytváranie zdrojového kódu však nebolo ich cieľom - slúžili najmä na navrhovanie postupnosti obrazoviek a na prezentáciu návrhu používateľom. Predpokladalo sa, že vlastná implementácia GUI vznikne manuálne na základe takto pripraveného prototypu.

Šiestu kategóriu predstavovali riešenia pre konkrétne cieľové platformy, ako sú webové aplikácie s dôrazom na funkčné prvky použiteľnosti (LIW) v článku „Automatic Method of Generating a Web Prototype Employing Live Interactive Widget to Validate Functional Usability Requirements“ alebo Android aplikácie s generovanými prezentačnými vrstvami podľa

anotovaných diagramov tried v článku „Model-driven GUI generation and navigation for Android BIS apps“. Tieto práce dokazovali, že anotácie v modeloch bolo možné účinne využiť na generovanie bohatého UI pre konkrétnu technológiu. Zároveň boli veľmi úzko viazané na danú platformu a často sa sústreďovali skôr na prototypovanie alebo generovanie špecifických komponentov než na plnú kostru produkčného kódu. [7] [8]

Osobitnú kategóriu tvorili prístupy, ktoré nadväzovali na formalizované UML modely a sústreďovali sa na analýzu a vizualizáciu výsledkov (napríklad systém MINERVA v článku „Visualization and Interpretation of Analysis Results within the Context of Formalized UML Diagrams“). [10]

Tie síce neprinášali priamo generovanie GUI kódu, ale ukázali dôležitosť toho, aby formálna zložka modelu zostávala previazaná s pôvodnými diagramami, táto myšlienka je pre nás dôležitá pri úvahách o zachovaní väzby medzi UML modelom v Enterprise Architecte a generovaným Java kódom.

3. Identifikované nedostatky a motivačné faktory pre naše riešenie

Z porovnania uvedených kategórií vyplýva niekoľko pozorovaní, ktoré priamo formujú náš návrh.

Po prvé, väčšina existujúcich riešení sa orientovala na generovanie prototypov používateľského rozhrania, nie na plnohodnotný zdrojový kód, ktorý by bolo možné okamžite skompilovať, spustiť a ďalej rozvíjať. V článkoch ako „Automatic Generation of User Interfaces from Domain and Use Case Models“ alebo „Generating User Interface Prototypes from Scenarios“ slúžilo generované používateľské rozhranie predovšetkým na validáciu požiadaviek, prepojenie modelu prípadov použitia s doménovým modelom a demonštráciu správania systému. Výsledkom boli prototypy, ktoré často nemali ambíciu stať sa základom produkčnej implementácie. To znamenalo, že po ukončení fázy prototypovania musel tím vytvoriť používateľské rozhranie na novo v cieľovom programovacom jazyku, čím sa strácala významná časť investície vložennej do modelovania. [1] [3]

Po druhé, väčšina prístupov sa sústreďovala na používateľské rozhranie a doménový model zostával buď implicitne predpokladaný, alebo sa k nemu pristupovalo oddelene. Aj keď článok „Automatic Generation of User Interfaces from Domain and Use Case Models“ explicitne prepájal doménový model s modelom používateľského rozhrania a s modelom prípadov použitia podporeným OCL, výsledkom bol opäť prototyp používateľského rozhrania. [1]

V praxi je však dôležité, aby z toho istého modelu vznikala nielen GUI vrstva, ale aj doménová vrstva v podobe tried, rozhraní a enumerácií v cieľovom jazyku. Inak hrozí, že UI prototyp a doménová implementácia sa budú vyvíjať v dvoch odlišných líniiach, čo komplikuje údržbu a zvyšuje riziko nekonzistencie. Našou ambíciou je generovať z jedného UML modelu v Enterprise Architecte súčasne doménové objekty aj Swing UI, čím dosiahneme jednotný a konzistentný základ aplikačnej architektúry v doménovej aj prezentačnej vrstve.

Po tretie, väčšina prístupov nekládla dôraz na presné zachovanie polohovania prvkov tak, ako boli rozmiestnené v grafických diagramoch v CASE nástroji. V UIDL prístupoch a BPMN transformáciách išlo predovšetkým o logickú štruktúru rozhrania a typy komponentov. Konkrétne súradnice a rozmery boli spravidla generované podľa generických layout algoritmov alebo ponechané na cieľovú technológiu. [5] [9]

V praxi však často požadujeme, aby prototyp zodpovedal vizuálnemu návrhu, ktorý analytik vytvoril v CASE nástroji - ak je napríklad tlačidlo zarovnané vpravo hore v diagrame používateľského rozhrania, očakáva sa, že v prototypovej aplikácii bude na tom istom mieste. Zachovanie tejto vizuálnej konzistencie medzi diagramom a vygenerovaným GUI je zároveň mimoriadne dôležité pri komunikácii so zákazníkom, ktorý sa orientuje vizuálne a nie prostredníctvom metamodelov. V riešení preto chceme zachovávať súradnice prvkov z EA diagramov a používať ich vo Swingu, aby mal prvok v diagrame a zodpovedajúci komponent v okne tú istú pozíciu.

Po štvrté, veľká časť existujúcich generátorov predpokladala, že model bude exportovaný z CASE nástroja (typicky vo formáte XMI alebo v špeciálnom jazyku založenom na XML) a následne spracovaný v samostatnom transformačnom prostredí, často založenom na MDA/MDD platforme či špecifických „engine-och“. Tento prístup je síce flexibilný z hľadiska kombinovania rôznych nástrojov, ale pre praktické použitie v tímoch znamená zvýšenú komplexitu - vývojár musí poznať exportné formáty, príslušné transformácie, ich konfiguráciu a nástrojový reťazec. Z pohľadu každodenného použitia je oveľa atraktívnejšie riešenie, v ktorom sa všetko deje „na jedno kliknutie“ priamo v prostredí CASE nástroja bez potreby manuálneho exportu a importu modelu. Naš plánovaný addin v Enterprise Architecte presne toto umožní: Používateľ spustí generovanie z menu EA a výsledkom je priamo vygenerovaný Java kód.

Po piate, mnohé prístupy zavádzali nový špecializovaný jazyk na opis UI, čo je prípad USXML či iných UIDL alebo vyžadovali, aby sa modely doplnili rozsiahlymi anotáciami a formálnymi špecifikáciami (napr. OCL, špecifické anotácie pre Android, LIW vzory). To však v praxi znamená, že analytici a návrhári musia ovládať nielen UML a cieľový programovací jazyk, ale aj ďalší modelovací jazyk a nástroje. Zvlášť problematické to je v prostredí, kde sú štandardne používané UML diagramy tried a diagramy používateľského rozhrania, ktoré by tím modeloval tak či tak kvôli implementácii. Zavedenie ďalšieho jazyka pre potreby generovania prototypu zvyšuje náklady na učenie sa, riziko nekonzistencie a zvyšuje množstvo práce, ktoré je potrebné na udržiavanie viacerých modelov v súlade. Naše riešenie sa takémuto prístupu vedome vyhýba: Dodatočné informácie potrebné na generovanie používateľského rozhrania a jeho správania (napríklad typ akcie tlačidla) plánujeme uchovávať len vo forme tagov priamo na UML elementoch v Enterprise Architecte. Tým sa zabezpečí, že všetky informácie potrebné pre generovanie zostanú v jednom modeli bez potreby vlastného jazyka.

Po šieste, v oblasti prepojenia modelov s formálnou analýzou je jasne viditeľná potreba zachovať súlad medzi modelom a vykonateľnými artefaktmi - či už ide o formálne špecifikácie alebo prototypy UI. [10]

Pre nás to znamená, že generovanie Java kódu z UML diagramov by malo byť navrhnuté tak, aby bola zachovaná jednoznačná väzba medzi prvkami v modeli a vygenerovanými triedami a komponentmi vo Swingu. Zároveň plánujeme, aby model v Enterprise Architecte slúžil ako východisko pre implementáciu, zatiaľ čo kód bude vždy len odvodeným artefaktom, ktorý je možné z tohto modelu podľa potreby opakovane generovať. Ak sa UML model v Enterprise Architecte zmení - či už z dôvodu novej požiadavky alebo spätnej väzby od zákazníka, zmena sa vykoná v UML, nie v kóde a generátor vytvorí novú verziu kódu. Tým sa minimalizuje riziko rozchodu modelu a implementácie, ktoré v prístupoch založených na jednorazovom generovaní prototypu a následnom ručnom programovaní GUI často vzniká.

Na základe týchto nedostatkov smerujeme k riešeniu, ktoré nebude generovať iba prototyp, ale priamo kompilovateľný zdrojový kód, zachová presné polohovanie prvkov podľa UML diagramov, bude realizované ako addín integrovaný priamo v prostredí Enterprise Architecta, z jedného UML modelu súčasne vytvorí doménovú aj používateľskú vrstvu a vystačí si so štandardnými UML diagramami tried a diagramami používateľského rozhrania bez potreby osvojovať si nový špecializovaný jazyk. Zároveň plánujeme, aby model v Enterprise Architecte slúžil ako východisko pre implementáciu, zatiaľ čo kód bude vždy len odvodeným artefaktom, ktorý bude možné z tohto modelu podľa potreby opakovane generovať.

4. Zdôvodnenie voľby technológií a spôsobu riešenia

Pri koncipovaní nášho riešenia sme zvažovali viacero kľúčových rozhodnutí - voľbu CASE nástroja, implementačného jazyka pre addin, cieľového programovacieho jazyka a GUI knižnice, spôsob integrácie do vývojového prostredia a tiež aj to, či zaviesť nový modelovací jazyk.

Enterprise Architect sme si zvolili ako zdrojové modelovacie prostredie preto, že ide o v praxi široko používaný UML CASE nástroj, v ktorom sa bežne modelujú diagramy tried aj diagramy používateľského rozhrania. V prostredí mnohých projektov tieto diagramy štandardne vznikajú v rámci vývojového procesu - diagramy tried ako základ doménového modelu a UI diagramy ako nástroj komunikácie s používateľom a zároveň ako návrhový podklad pre implementáciu používateľského rozhrania v reálnej aplikácii. Naším cieľom je priamo z týchto existujúcich diagramov generovať prototyp aj kód, bez toho, aby bolo nutné modelovať ďalšie samostatné modely v ďalších jazykoch. Tým minimalizujeme dodatočnú prácu, ktorú musí tím vynaložiť len preto, aby získal prototyp. V porovnaní s prístupmi založenými na USIXML a podobných jazykoch tým odpadá potreba učiť sa ďalší jazyk, osvojiť si nové nástroje a udržiavať viaceré modely konzistentné.

Ako implementačný jazyk addinu sme zvolili C#, pretože Enterprise Architect poskytuje rozsiahle API prístupné cez .NET, ktoré umožňuje addinom zachytávať a obsluhovať udalosti súvisiace s interakciou používateľa s nástrojom Enterprise Architect a prístupovať k vnútornej štruktúre modelu - balíkom, diagramom, prvkom a hodnotám uloženým v tagoch, ako aj k ďalším súvisiacim informáciám. [11]

Z architektonického hľadiska je prirodzené situovať generátor ako plugin, ktorý je priamo pri zdroji modelu, podobne ako MagicDraw využil vlastný plugin mechanizmus pri integrácii GUI prototypovania. [4]

Tým sa vyhneme potrebe exportovať model do XMI a spúšťať externé transformácie v ďalšom nástroji. Používateľ v Enterprise Architecte jednoducho aktivuje náš addin cez položku v menu a výsledkom bude vygenerovaný kód, ktorý je možné okamžite skompilovať a spustiť. Toto jednodukové správanie zvyšuje praktickú použiteľnosť riešenia v reálnych tímoch.

Ako cieľový programovací jazyk volíme Javu a ako GUI knižnicu Java Swing. Táto voľba nadväzuje na prácu „Designing GUI Components from UML Use Cases“, kde boli transformačné

pravidlá formulované priamo pre Swingové komponenty („frame”, „button”, „label”, „text area”) a kde sa jasne ukázalo, že Swing poskytuje priamu korešpondenciu medzi UML prvkami a konkrétnymi grafickými ovládacími prvkami používateľského rozhrania. [2]

Zároveň vychádzame zo skutočnosti, že Java je stále silne zastúpená v podnikových informačných systémoch a Swing sa vo veľkej miere využíva v desktopových klientskych aplikáciách.

Z hľadiska modelovacích vstupov vychádzame z toho, že na vstupe budú štandardné UML diagramy tried a diagramy používateľského rozhrania, tak ako sa bežne modelujú v projektoch. Diagramy tried budú predstavovať doménový model, z ktorého budeme generovať Java triedy, rozhrania a enumerácie, zatiaľ čo diagramy používateľského rozhrania budú poskytovať informácie o obrazovkách, komponentoch a ich presnom rozmiestnení. Zachovanie presnej polohy prvkov považujeme za kľúčové: Cieľom je, aby prvok v Enterprise Architecte mal tú istú polohu ako zodpovedajúci komponent vo vygenerovanom Swing kóde. Tým vznikne stav, v ktorom môže analytik v prototypovej aplikácii priamo pozorovať realizáciu svojho diagramu. Významným dôsledkom tejto voľby je, že pre účely prototypovania nemusíme zavádzať vlastný jazyk. Analytik použije len tie diagramy, ktoré by sa v projekte modelovali aj bez generovania: Diagramy tried a diagramy používateľského rozhrania. Dodatočné informácie o správaní a špecifikách obrazoviek budeme zaznamenávať vo forme hodnôt v tagoch priamo na UML elementoch. Tým pádom zostávajú všetky potrebné významové informácie sústredené v jednom modeli a nie je nutné učiť sa nový jazyk ani používať ďalší nástroj.

Dôležité je, že z toho istého UML modelu budeme generovať súčasne doménovú vrstvu aj vrstvu používateľského rozhrania. Diagramy tried v Enterprise Architecte budú zdrojom pre generovanie Java tried, rozhraní a enumerácií reprezentujúcich doménové entity, zatiaľ čo UI diagramy budú zdrojom pre generovanie Swing obrazoviek, komponentov a navigácie. Tým sa zabezpečí, že doménová aj prezentačná časť aplikácie vzniknú z jedného zdroja a budú typovo a logicky konzistentné. V porovnaní s prístupmi, ktoré generujú len UI alebo pracujú s paralelnými modelmi to výrazne znižuje riziko nekonzistencie medzi doménou a rozhraním.

Výberom formy addinu v C# a generovaním Java Swing kódu chceme dosiahnuť, aby výsledný prototyp nebol len dočasným artefaktom na validáciu požiadaviek, ale aby poskytoval pevný základ pre ďalší vývoj.

Pri výbere CASE nástroja sme okrem EA zvažovali aj MagicDraw a Visual Paradigm, keďže oba nástroje poskytujú rozhrania na prácu s prvkami UML modelu. Pre naše riešenie však bolo rozhodujúce, aby sme vedeli addin vyvíjať priamo v jazyku .NET a zároveň mali priamy prístup k UML prvkom aj k ich vlastnostiam a doplnkovým informáciám (napr. vzťahy, stereotypy, hodnoty uložené v tagoch) v rozsahu potrebnom pre naše spracovanie. Enterprise Architect umožňuje prístup k repozitáru modelu cez COM rozhranie, ktoré je možné priamo využiť v C#. V prípade MagicDraw a Visual Paradigm je prístup k modelu cez rozhrania nástroja riešený cez ich vlastné pluginové mechanizmy, preto by pri našom technologickom zámere vyžadoval dodatočné integračné riešenia a zvýšil by implementačnú náročnosť, preto sme si vybrali EA. Ďalej sme zvažovali StarUML a Astah UML, no tieto nástroje neposkytujú rozhranie, cez ktoré by sme vedeli v potrebnom rozsahu pristupovať k prvkom UML modelu a pracovať s ich vlastnosťami (napr. stereotypy, hodnoty uložené v tagoch a väzby), čo je nevyhnutné pre naše spracovanie a udržiavanie prepojení medzi modelom a generovanými artefaktmi. Pri voľbe implementačného jazyka addinu sme okrem C# zvažovali aj VB.NET a F#, ktoré sú použiteľné aj s EA interop knižnicou, avšak C# je najvhodnejší vzhľadom na štandardné postupy vývoja COM a .NET rozšírení, dostupnosť knižníc a dlhodobú udržateľnosť riešenia. Jazyky Java, Python, JavaScript a TypeScript sme zamietli okamžite, keďže neumožňujú priame napojenie na EA COM rozhranie a prístup k prvkom EA by si preto vyžadoval sprostredkujúcu integračnú vrstvu. Ako cieľovú technológiu pre výslednú aplikáciu sme popri Java so Swingom zvažovali C# (WPF), Python (PyQt), C++ (Qt) a alternatívne UI riešenie v rámci Javy cez JavaFX. C# sme nevybrali, keďže by výsledná aplikácia bola viazaná na platformu Windows, čím by sa znížila jej použiteľnosť na iných operačných systémoch. Python sme nezvolili, pretože by si vyžadoval použitie externého GUI frameworku, ktorý nie je súčasťou štandardného prostredia, čo pre náš účel nebolo vhodné. Jazyk C++ sme nezvolili, keďže by znamenal výrazne vyššiu implementačnú náročnosť pri správe knižníc a distribúcii naprieč platformami v porovnaní s Javou. Technológiu JavaFX sme nevybrali, keďže v cieľových prostrediach nie je dostupná v rámci štandardného prostredia Javy a jej použitie by vyžadovalo doplnenie samostatných JavaFX knižníc. Swing je štandardnou súčasťou Javy.

Existujúce nástroje

Na overenie, či už existuje hotové riešenie sme si nainštalovali a prakticky otestovali nástroje na modelom riadené generovanie. Ukázalo sa, že buď generujú kód z tried alebo vytvárajú UI prototypy bez väzby na UML doménu a hlavne nedokážu spoľahlivo preniesť presné rozloženie prvkov z UI diagramov do výsledného GUI.

1. AndroMDA

AndroMDA je MDA framework, ktorý načíta UML model (vo formáte XMI) a generuje kód na základe stereotypov a tzv. „cartridges“ (sady šablón a pravidiel). Aj keď AndroMDA cez niektoré „cartridge“ dokáže generovať časti prezentačnej vrstvy z UML modelu, vychádza z logických UML prvkov. To znamená, že generovanie pracuje so štruktúrou a tokmi, nie s informáciami o konkrétnom rozložení komponentov na obrazovke, ktoré sú uložené v UI diagrame v EA. [18]

2. Papyrus Moka

Tento prístup je postavený na tom, že sa UML model dá spúšťať a overovať z pohľadu jeho správania, nie na generovaní finálneho UI podľa návrhu obrazoviek. V praxi to znamená, že rieši beh modelu (logiku), kým informácie potrebné na určenie rozmiestnenia UI prvkov na obrazovke nie sú súčasťou vstupného modelu. [19] [20]

3. Integranova Model Execution System

OO-Method je metodika konceptuálneho modelovania a Integranova MES predstavuje k nej kompletne softvérové prostredie, ktoré z konceptuálnych modelov automaticky vytvára databázovú vrstvu, aplikačnú logiku aj používateľské rozhranie. Používateľské rozhranie je v tomto prístupe popísané prezentačným modelom založeným na všeobecných prvkoch obrazovky a pravidlách interakcie, ktoré sú navrhnuté tak, aby boli nezávislé od konkrétnej platformy. Z tohto dôvodu sa pracuje so všeobecným opisom obrazoviek a interakcií, nie s presnými parametrami rozmiestnenia prvkov na obrazovke. Pre náš cieľ je to nevýhoda, pretože potrebujeme prevziať rozloženie prvkov priamo z EA User Interface diagramov, pričom tento typ informácie sa v OO-Method používa v inej

forme - rozhranie sa opisuje konceptuálne a výsledné rozmiestnenie prvkov sa určuje až v rámci generovania pre cieľovú technológiu. [21] [22]

4. **Cameo Simulation Toolkit**

Cameo Simulation Toolkit sa inštaluje do prostredia MagicDraw a slúži na simulovanie správania UML a SysML modelov, teda na prehrávanie toho, ako sa model vykonáva spolu s vizuálnym zobrazením priebehu. Nástroj umožňuje pracovať aj s jednoduchými formulármi určenými na zadávanie hodnôt a spúšťanie simulácie. Pri našom ciele nepomáha, lebo rieši beh modelu, nie generovanie kódu z diagramov. [23] [24] [25]

5. **WebRatio Platform**

Jazyk IFML rieši modelovanie interakcií a navigácie používateľského rozhrania na koncepcnej úrovni (obrazovky, toky medzi nimi). Tento typ modelu je vhodný na opis štruktúry a prechodov, ale pri našom ciele je rozhodujúce, aby vstup obsahoval aj informácie o umiestnení prvkov z UI diagramu v EA a aby sa tieto údaje dali použiť pri generovaní kódu. [26] [27]

6. **Visual Paradigm**

Visual Paradigm podporuje import XMI súboru (exportovaného z EA) a poskytuje generovanie zdrojových súborov z UML diagramu tried. Súčasne má samostatnú podporu pre wireframy a storyboardy ako prezentačné výstupy. Generovanie kódu je viazané na model tried, UI diagramy nie sú vstupom, z ktorého by vznikol UI kód prepojený s modelom tried. [28]

7. **Eclipse EMF**

EMF generovanie je postavené na Ecore modeli, ktorý opisuje štruktúru domény a z neho vzniká výsledný zdrojový kód. Tento prístup rieši model tried (a jeho väzby), ale neposkytuje prostriedky na modelovanie obrazoviek, takže generovanie UI z diagramov obrazoviek nie je týmto prístupom realizovateľné. [29] [30] [31]

8. **Acceleo**

Acceleo umožňuje generovať zdrojové súbory podľa šablón nad modelom. XMI opisuje UML prvky a ich vlastnosti, nie presné grafické rozmiestnenie prvkov v diagrame. Z pohľadu generovania UI je to kritické: Ak vstup nenesie súradnice UI komponentov, šablóna ich nevie korektne odvodiť. [32] [33] [34]

9. **PyUiBuilder**

PyUiBuilder je vizuálny nástroj na tvorbu GUI, ktorý generuje Python kód. Nejde o UML modelovanie: Nevzniká tu model tried v UML ani väzby UI prvkov na UML prvky. Je to samostatný návrh obrazoviek. Nevie zabezpečiť konzistenciu medzi doménovým modelom a UI. [35]

10. **Anima**

Anima generuje kód (React, HTML, Tailwind a pod.) priamo z dizajnu vo Figma. Figma však neobsahuje UML doménový model, takže výsledný UI kód nevie byť automaticky konzistentný s doménou a vyžaduje ďalšie ručné dopracovanie. [36] [37]

11. **Builder.io**

Builder.io je nástroj zameraný na prevod dizajnu na webový kód (React). Tento prístup nerieši väzby na UML diagramy tried. [38] [39]

12. **TeleportHQ**

TeleportHQ je nástroj zameraný na prevod dizajnu na webový kód (HTML, CSS). Tento prístup nerieši väzby na UML diagramy tried. [40] [41]

13. **GUI dizajnéry (JFormDesigner, WindowBuilder, NetBeans GUI Builder)**

Tieto nástroje umožňujú vizuálne naklikáť GUI a následne vygenerovať Java Swing kód. UI je však navrhované mimo UML modelu, bez väzby na diagramy tried a bez možnosti preklikávať sa medzi obrazovkami. [42] [43] [44] [45] [46]

Vstavané generovanie - Enterprise Architect

V EA sme otestovali existujúce generovanie kódu z modelu tried. Výstup vzniká z tried, rozhraní a enumerácií, ale v praxi sa ukázalo, že tento spôsob je použiteľný iba ako hrubý základ: Ak názov prvku obsahoval medzeru, generátor ju preniesol do výstupu bez transformácie a zároveň sa do výsledného kódu nepremietali vzťahy medzi prvkami tak, aby bol výsledok priamo použiteľný bez ručného zásahu.

Následne sme skúšali generovanie pre diagramy obrazoviek - User Interface diagram, UserInterfaceWin32 diagram, Apple Wireframe diagram, Android Wireframe diagram, Dialog Wireframe diagram, Webpage Wireframe diagram a Windows Phone Wireframe diagram. Vstavané generovanie nefungovalo pre ani jeden z týchto diagramov.

Rozhodujúci rozdiel je v tom, že EA má informáciu o polohe prvkov uloženú priamo v repozitári. Objekt diagramu je mapovaný do databázových tabuliek repozitára. Nerobíme XMI exporty, pretože XMI v UML exporte neuchováva súradnice diagramových prvkov. Rozloženie obrazovky preto získavame z údajov uložených v EA repozitári.

Kľúčová výhoda nášho navrhovaného riešenia je, že pracuje s jedným EA modelom: Z toho istého repozitára vieme spracovať modely tried aj User Interface diagramy. Z UI diagramov budeme čítať súradnice prvkov a súčasne budeme zachovávať väzby UI prvkov na prvky diagramov tried. Výsledkom bude vygenerovaný kód, v ktorom budú triedy a UI navzájom prepojené konzistentne, bez ručného dopĺňania mapovaní a bez straty rozloženia prvkov.

5. Stručný opis plánovaného spôsobu riešenia

Na základe analýzy existujúcich prístupov navrhujeme riešenie, ktoré rozširuje nástroj Enterprise Architect o addin implementovaný v jazyku C#. Tento addin bude využívať API nástroja Enterprise Architect, ktoré umožňuje priamy prístup k repozitáru modelu, balíkom, diagramom, prvkom, konektorom a hodnotám uloženým v tagoch a používateľovi bude k dispozícii prostredníctvom vlastnej položky v menu nástroja. Po spustení generovania z menu Enterprise Architectu bude addin nad aktuálnou verziou modelu vytvárať Java projekt, ktorý bude obsahovať doménové objekty odvodené z diagramov tried a grafické používateľské rozhranie implementované pomocou knižnice Swing, odvodené z diagramov používateľského rozhrania.

Konceptuálne budeme rozlišovať dve hlavné vetvy generovania: Vetvu generovania doménových objektov a vetvu generovania obrazoviek používateľského rozhrania. V oboch prípadoch budeme vychádzať z toho istého UML modelu v Enterprise Architecte, pričom doménové balíky budú obsahovať triedy, rozhrania a enumerácie a balíky používateľského rozhrania budú obsahovať obrazovky a prvky UI. Addin predpokladá, že balíková štruktúra v Enterprise Architecte zodpovedá logickej štruktúre cieľových Java balíkov, preto pri generovaní vytvorí adresárovú štruktúru projektu tak, aby názvy balíkov v UML zodpovedali deklaráciám „package“ v generovanom kóde. Tým sa zabezpečí, že štruktúra modelu a štruktúra projektu budú navzájom jasne zodpovedať.

a. Vetva doménových objektov

Vo vetve doménových objektov budeme prechádzať vybrané balíky, v ktorých sa budú nachádzať UML elementy typu trieda, rozhranie a enumerácia. Pre každý takýto element vytvoríme samostatný Java súbor, ktorý bude obsahovať deklaráciu balíka odvodenú z názvu balíka v EA, potrebné importy (odvodené z typov použitých v atribútoch a vzťahoch pri triedach a z návratových typov a parametrov metód pri rozhraniach) a následne deklaráciu triedy, rozhrania alebo enumu odvodenú z názvu UML elementu. Pri generovaní importov budeme analyzovať všetky typy, ktoré sa v danom elemente používajú - typy atribútov, typy nadradených tried, typy rozhraní uvedených v realizáciách a typy tried, na ktoré vedú asociácie. Z týchto typov zostavíme množinu Java balíkov, v ktorých sa nachádzajú a pre balíky vygenerujeme deklarácie import. Tým sa zabezpečí, že každý vygenerovaný súbor bude obsahovať importy, bez potreby ich ručného

dopĺňania. Samotný názov Java triedy, rozhrania alebo enumu bude priamo odvodený z názvu UML elementu, takže medzi modelom a vygenerovaným kódom vznikne jednoznačná a ľahko sledovateľná korešpondencia. Pri triedach sa z UML atribútov vytvoria inštančné premenné v Jave, pričom sa uplatní mapovanie z UML typov na typy jazyka Java a zohľadní sa, či typ atribútu pochádza z iného balíka, čo si vyžiada doplnenie importov. Generalizácie medzi triedami zistené z konektorov typu „generalization“ sa premietnu do dedičnosti: Ak trieda v UML dedí od inej triedy (obr. 10, obr. 12, obr. 15, obr. 16, obr. 17), v deklarácii vygenerovaného Java kódu bude uvedená príslušná nadradená trieda. V prípadoch, keď trieda v UML realizuje rozhranie (obr. 15, obr. 16, obr. 17), budeme v deklarácii vygenerovanej triedy dopĺňať „implements“ s príslušným rozhraním a pre každú metódu definovanú v tomto rozhraní vytvoríme v triede zodpovedajúcu deklaráciu metódy s rovnakým názvom, návratovým typom a parametrami. Tieto metódy budú vo vygenerovanom kóde predstavovať „override“ metód definovaných v rozhraní. Vývojár následne do tela týchto metód doplní vlastnú implementáciu, takže kostra triedy bude pripravená na postupné dopĺňanie aplikačnej logiky.

Asociácie medzi triedami budú posudzované z hľadiska kardinalít na oboch koncoch konektora. V prípadoch, keď kardinalita na asociácii umožňuje, aby bola jedna inštancia triedy prepojená s viacerými inštanciami druhej triedy (napríklad 0..*, 1..* alebo iný interval s hornou hranicou väčšou ako 1), vygenerujeme v danej triede atribút typu zoznam objektov druhej triedy (obr. 9, obr. 13). Zároveň pre tento zoznam vygenerujeme prístupové metódy - metódu „getter“, ktorá vracia celý zoznam súvisiacich objektov a zodpovedajúcu metódu „setter“, ktorá umožňuje tento zoznam ako celok nastaviť. V prípadoch, keď je kardinalita ohraničená na najviac jednu inštanciu, vygenerujeme v danej triede atribút typu druhej triedy spolu s „getterom“ a „setterom“ na prístup k tejto väzbe. Týmto spôsobom sa vzťahy medzi triedami v UML (mená tried, typy asociácií a multiplicity) priamo odrazia vo vygenerovanom Java kóde v podobe odkazov na jediný objekt (pri kardinalite najviac 1) a zoznamov (pri kardinalite väčšej než 1). Pre enumerácie budeme z elementov typu „Enumeration“ generovať Java „enumy“ (obr. 10, obr. 11, obr. 12, obr. 14). Názvy jednotlivých hodnôt v týchto „enumoch“ preberieme priamo z UML modelu, aby bolo možné v doménovom kóde používať jednoznačne pomenované „enum“ hodnoty a nebolo potrebné tieto hodnoty ručne prepisovať.

b. Vetva používateľského rozhrania

Vo vetve používateľského rozhrania predpokladáme, že v Enterprise Architecte budú existovať balíky vyhradené pre obrazovky a prvky UI. Každá obrazovka bude reprezentovaná UML elementom, ktorý sa bude interpretovať ako budúca Java trieda reprezentujúca okno alebo dialóg aplikácie. Na úrovni modelu budeme predpokladať tag „screenType“, ktorého hodnota určí, či sa má vygenerovaná trieda správať ako hlavné aplikačné okno (okno typu „JFrame“) alebo ako dialóg („JDialog“). Podľa toho sa pri generovaní nastaví príslušná nadradená trieda zo Swingu. Tag „isMainScreen“ budeme používať na označenie hlavnej obrazovky aplikácie, ktorú po spustení inicializuje a zobrazí trieda obsahujúca metódu „main“. Ako názov vygenerovanej Java triedy obrazovky použijeme názov UML triedy, ktorá je s obrazovkou prepojená konektorom typu „Realization“ (názov sa následne upraví do formátu vhodného pre Javu) - obr. 2, obr. 5, obr. 7. Alias obrazovky použijeme ako názov okna vo vygenerovanej aplikácii. Týmto prístupom oddelíme názov triedy používaný v kóde od textu zobrazovaného používateľovi.

Jednotlivé prvky používateľského rozhrania budú reprezentované UML elementmi na diagrame obrazovky, pričom ich typ bude odvodený zo „stereotype“ (napríklad rozlíšenie textového popisu, vstupného poľa, tlačidla, zaškrťavacieho políčka, „comboboxu“, tabuľky, dátumového komponentu alebo panelu). Pre každý takýto prvok vytvoríme vnútornú reprezentáciu, ktorá bude okrem odkazu na pôvodný element obsahovať aj informáciu o type komponentu (jeho stereotyp) a zároveň prevezme jeho diagramové súradnice („top“, „left“, „bottom“, „right“), z ktorých v kóde (spolu so súradnicami obrazovky, na ktorej sa nachádza) vypočítame polohu, rozmery komponentu a jeho veľkosť v rámci obrazovky (súradnice prevedieme zo súradníc diagramu na súradnice v rámci obrazovky). Pri generovaní kódu použijeme tieto súradnice na priame umiestnenie komponentov na obrazovku bez automatického rozloženia. Veľkosť okna potom nastavíme tak, aby sa doň zmestili všetky prvky. Tým dosiahneme, že rozloženie prvkov vo vygenerovanom okne bude vizuálne zodpovedať rozloženiu prvkov v diagrame v Enterprise Architecte a poloha komponentov v modeli a vo výslednej aplikácii bude navzájom konzistentná.

Pri generovaní textov zobrazovaných používateľovi budeme vychádzať priamo z vlastností „Alias“ a „Name“ UML elementov. Alias obrazovky použijeme ako názov okna vo vygenerovanej aplikácii. Pri prvkoch typu „label“ sa ako text popisku použije prednostne alias (ak nebude

vyplnený, použije sa názov elementu) - obr. 3, obr. 4, obr. 6, obr. 8. Pri tlačidlách sa alias použije ako text tlačidla, pričom pri tlačidlách s ikonou sa alias použije ako nápoveda („tooltip“). Ak sa ikonu nepodari načítať, alias sa nastaví ako meno komponentu na jeho identifikáciu (obr. 1, obr. 2, obr. 3, obr. 4, obr. 5, obr. 6, obr. 7, obr. 8). Alias bude slúžiť aj ako nápoveda pri textovom poli, kde sa (ak bude vyplnený) nastaví ako „tooltip“ (obr. 3, obr. 6, obr. 8). Pri „comboboxoch“ sa text položiek odvodí z aliasov literálov enumerácie, ktorá je s komponentom prepojená konektorom „Realization“, aby boli zobrazené hodnoty používateľsky zrozumiteľné a zároveň priamo odvodené z enumerácií v UML modeli (obr. 3, obr. 6, obr. 8). Texty používateľského rozhrania budú definované priamo v EA modeli, takže nebude potrebné vytvárať ďalší samostatný zoznam textov mimo modelu.

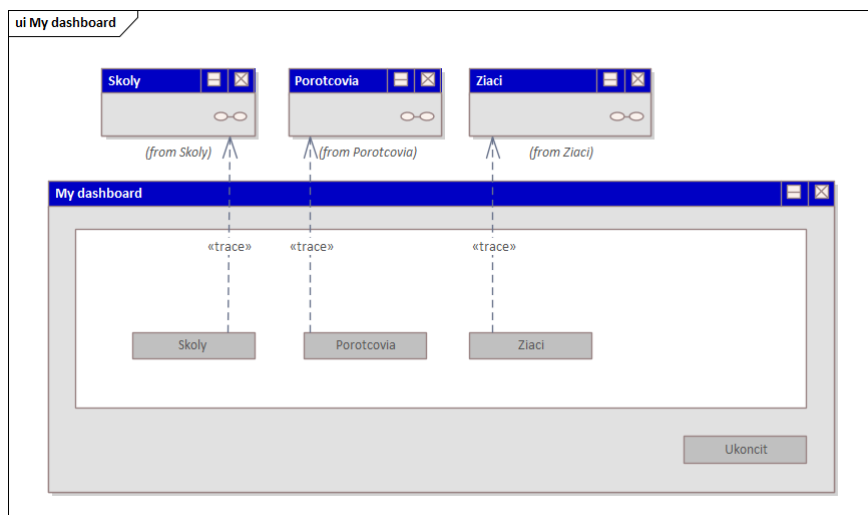
Osobitnú pozornosť budeme venovať generovaniu tabuliek. Pri prvku so stereotypom „table“ predpokladáme, že bude v EA modeli prepojený konektorom typu „Realization“ s doménovou triedou (táto trieda bude na konektore na strane „Supplier“), ktorej inštancie majú byť v tabuľke reprezentované (obr. 2, obr. 5, obr. 7). Názvy stĺpcov nebudeme brať automaticky zo všetkých atribútov, ale iba z tých atribútov, ktoré budú mať vyplnený tag „tableName“ - hodnota tohto tagu sa použije ako používateľsky zrozumiteľný názov stĺpca. Okrem toho generátor prejde aj konektory tejto doménovej triedy a rovnakým spôsobom doplní stĺpce aj z atribútov ďalších pripojených tried (opäť iba tam, kde bude vyplnený „tableName“). Na základe toho sa vytvorí v kóde pole názvov stĺpcov, ktoré sa použije pri inicializácii tabuľkového komponentu a tabuľku umiestni do posuvného panelu tak, aby bolo možné pracovať aj s väčším množstvom riadkov.

Pri tlačidlách budeme popri textovom popise využívať aj informáciu o type akcie, ktorá bude uložená v tagu „actionType“ (obr. 1, obr. 2, obr. 3, obr. 4, obr. 5, obr. 6, obr. 7, obr. 8). Na základe hodnoty tohto tagu rozlíšime základné typy správania bez toho, aby bolo potrebné ručne písať kód pre reakcie po stlačení. Ak bude tag určovať zatvorenie aktuálneho okna, vygenerujeme spracovanie kliknutia, ktoré po stlačení tlačidla zavrie príslušné okno. Ak bude hodnota tagu zodpovedať ukončeniu celej aplikácie, vygenerujeme spracovanie kliknutia, ktoré pri aktivácii tlačidla ukončí beh aplikácie. V prípade, že tag bude určovať otvorenie inej obrazovky, generátor podľa konektora so stereotypom „trace“ v UML modeli nájde cieľovú obrazovku a pri kliknutí na tlačidlo vytvorí a zobrazí novú inštanciu Java triedy vygenerovanej pre túto obrazovku. Tlačidlá, pri ktorých tag typu akcie nebude vyplnený, budú vygenerované len ako grafické komponenty bez

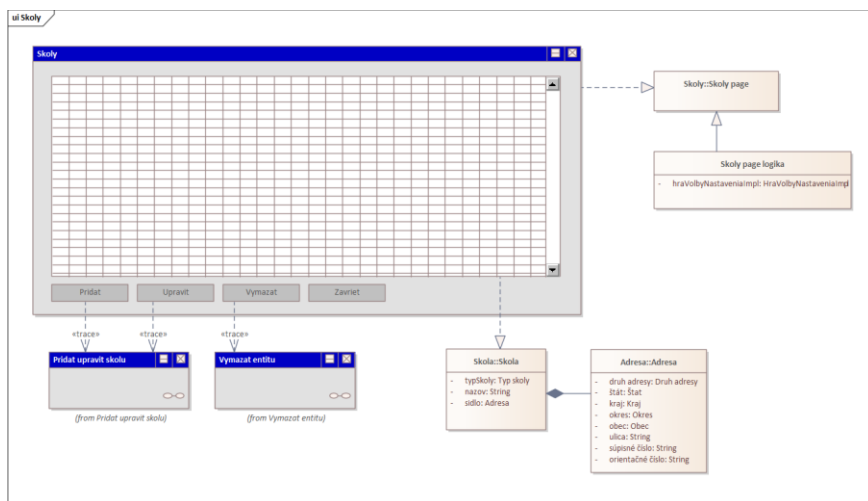
obslužnej logiky, pripravené na ručné doplnenie špecifického správania v nasledujúcich fázach vývoja.

Okrem tried reprezentujúcich jednotlivé obrazovky budeme generovať aj hlavnú spúšťačiu triedu, ktorá bude slúžiť ako vstupný bod aplikácie. Táto trieda vytvorí inštanciu obrazovky označenej v modeli tagom „isMainScreen“ ako hlavná a zabezpečí jej zobrazenie používateľovi. Pri spustení aplikácie nastavíme, aby zatvorenie hlavného okna ukončilo aplikáciu a aby sa hlavná obrazovka zobrazila vycentrovaná na obrazovke. Všetky väzby medzi triedami, enumeráciami a prvkami používateľského rozhrania, ktoré sú v modeli vyjadrené konektormi (napr. prepojenia na „enumy“, väzby medzi obrazovkami) sa premietnu do vygenerovaného kódu tak, aby štruktúra výslednej aplikácie zodpovedala štruktúre UML modelu. Tým vznikne plne spustiteľná aplikácia, ktorej doménová aj používateľská vrstva budú generované z jedného UML modelu v Enterprise Architecte a ktorá bude odrážať štruktúru diagramov tried aj UI diagramov. Diagramy tried a diagramy používateľského rozhrania budú navzájom zosúladené tak, aby spolu tvorili jednotný návrh aplikácie. Vo vygenerovanom kóde budú triedy a prvky používateľského rozhrania navzájom prepojené tak, aby aplikácia zodpovedala väzbám definovaným v UML modeli.

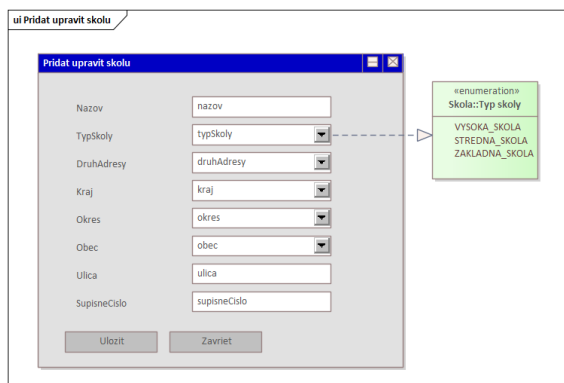
Príklad namodelovaného vstupu v EA, z ktorého naše plánované riešenie vygeneruje zdrojový kód prototypu:



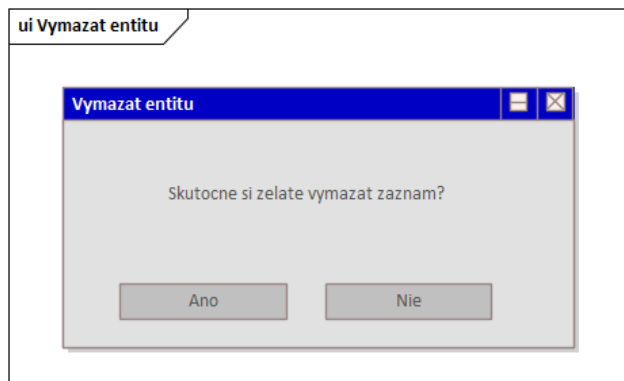
Obr. 1: UI diagram obrazovky „My dashboard“



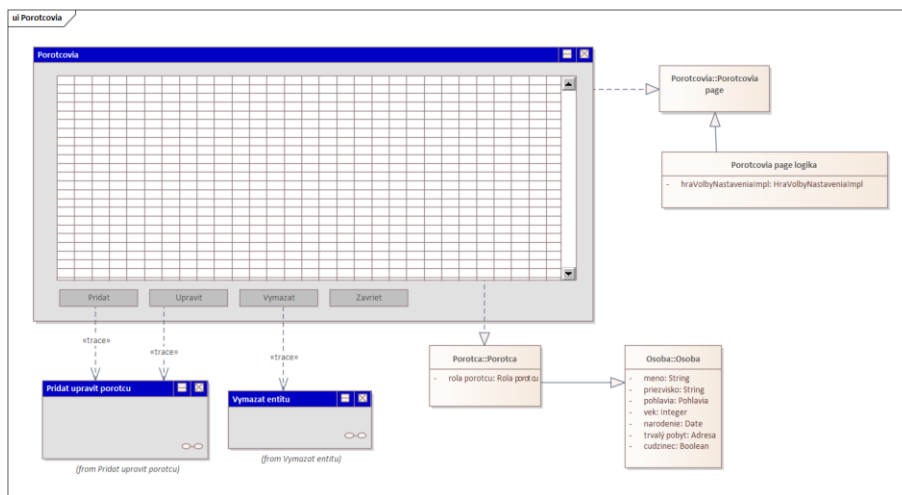
Obr. 2: UI diagram obrazovky „Skoly“



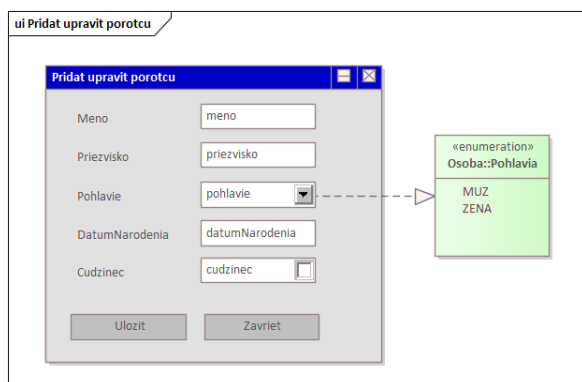
Obr. 3: UI diagram obrazovky „Pridat upravu skoly“



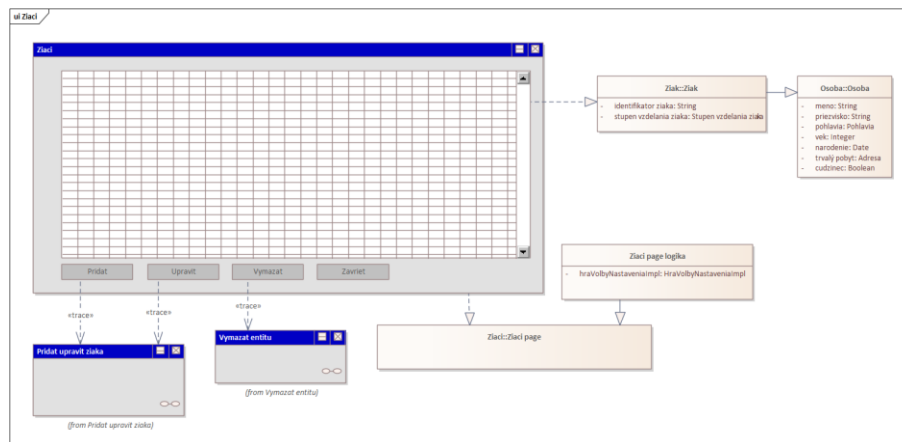
Obr. 4: UI diagram obrazovky „Vymazat entitu“



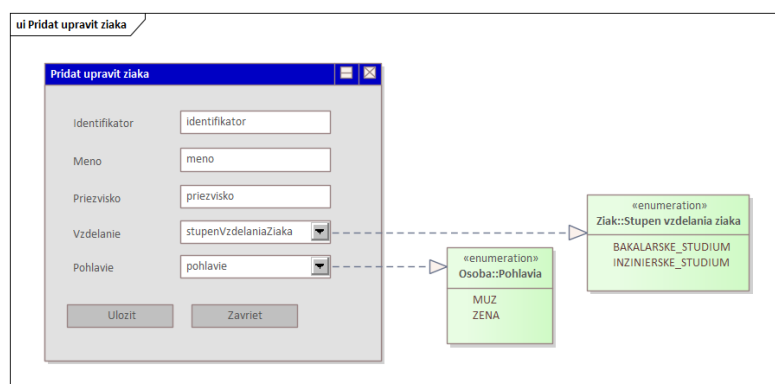
Obr. 5: UI diagram obrazovky „Porotcovia“



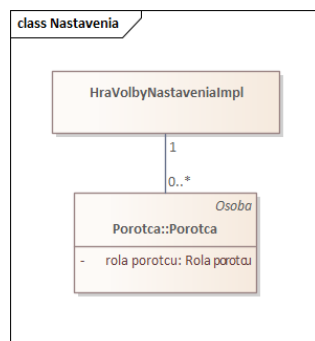
Obr. 6: UI diagram obrazovky „Pridat upravit porotcu“



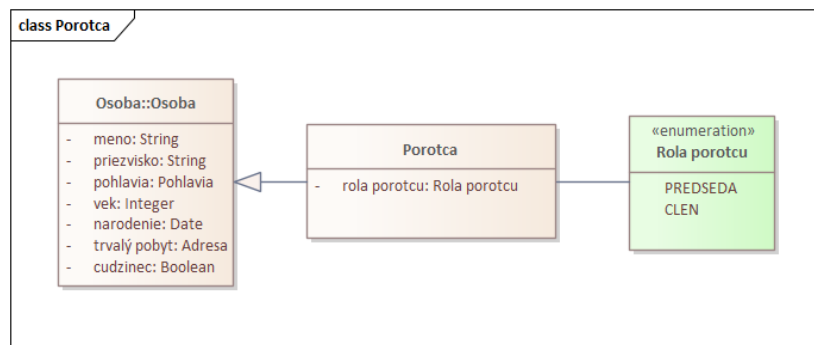
Obr. 7: UI diagram obrazovky „Ziaci“



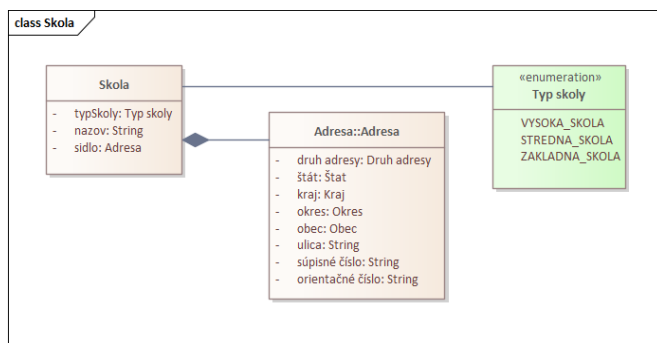
Obr. 8: UI diagram obrazovky „Pridat upraviti ziala“



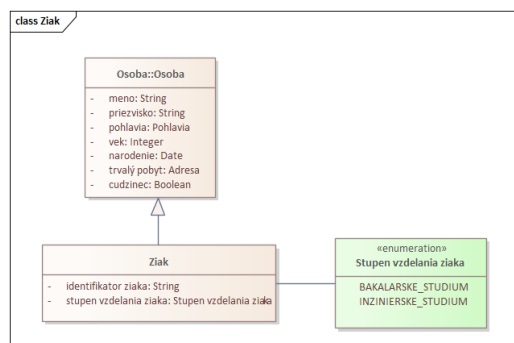
Obr. 9: Diagram tried modulu „Nastavenia“



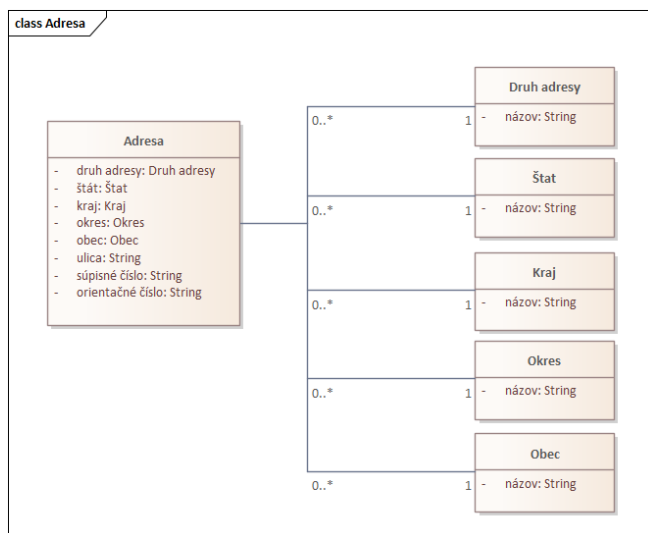
Obr. 10: Diagram tried modulu „Porotca“



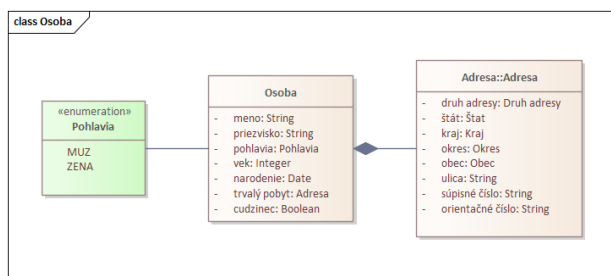
Obr. 11: Diagram tried modulu „Skola“



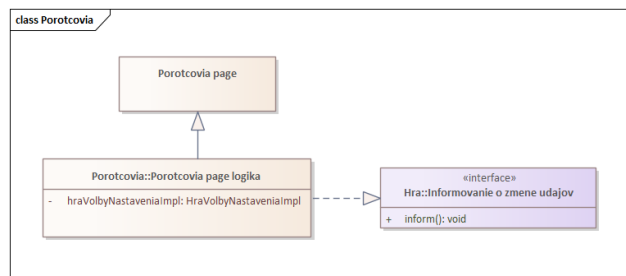
Obr. 12: Diagram tried modulu „Ziak“



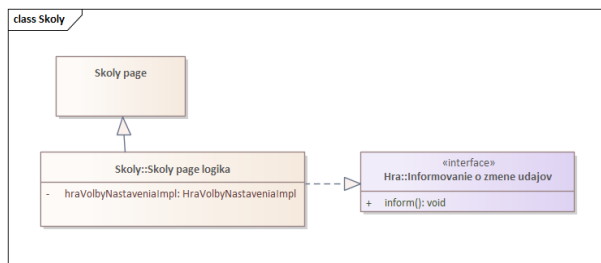
Obr. 13: Diagram tried modulu „Adresa“



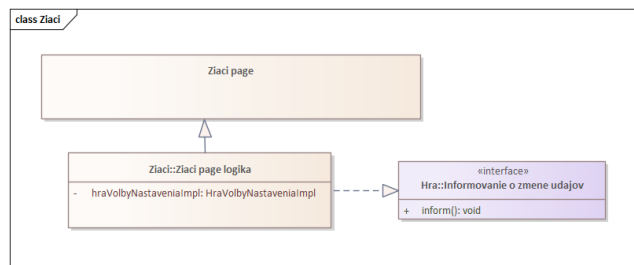
Obr. 14: Diagram tried modulu „Osoba“



Obr. 15: Diagram tried modulu „Porotcovia“

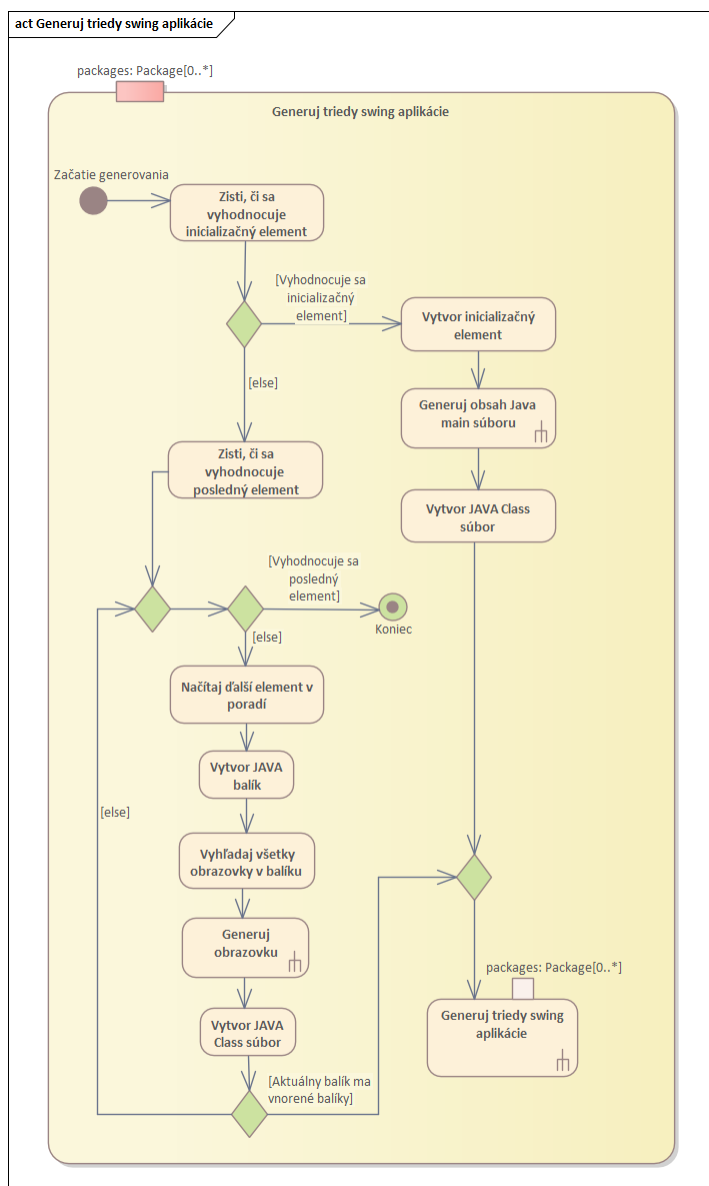


Obr. 16: Diagram tried modulu „Skoly“

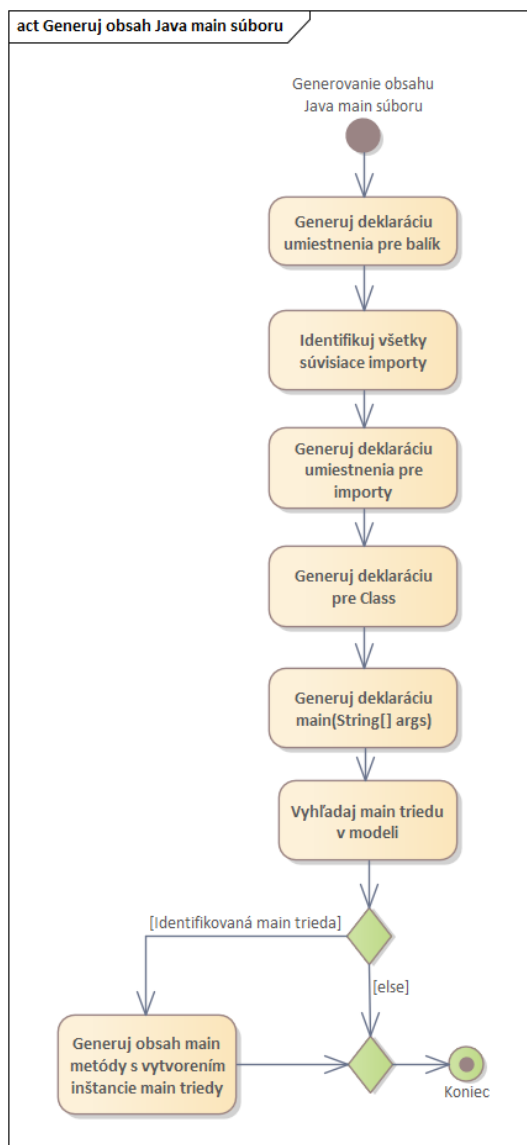


Obr. 17: Diagram tried modulu „Ziaci“

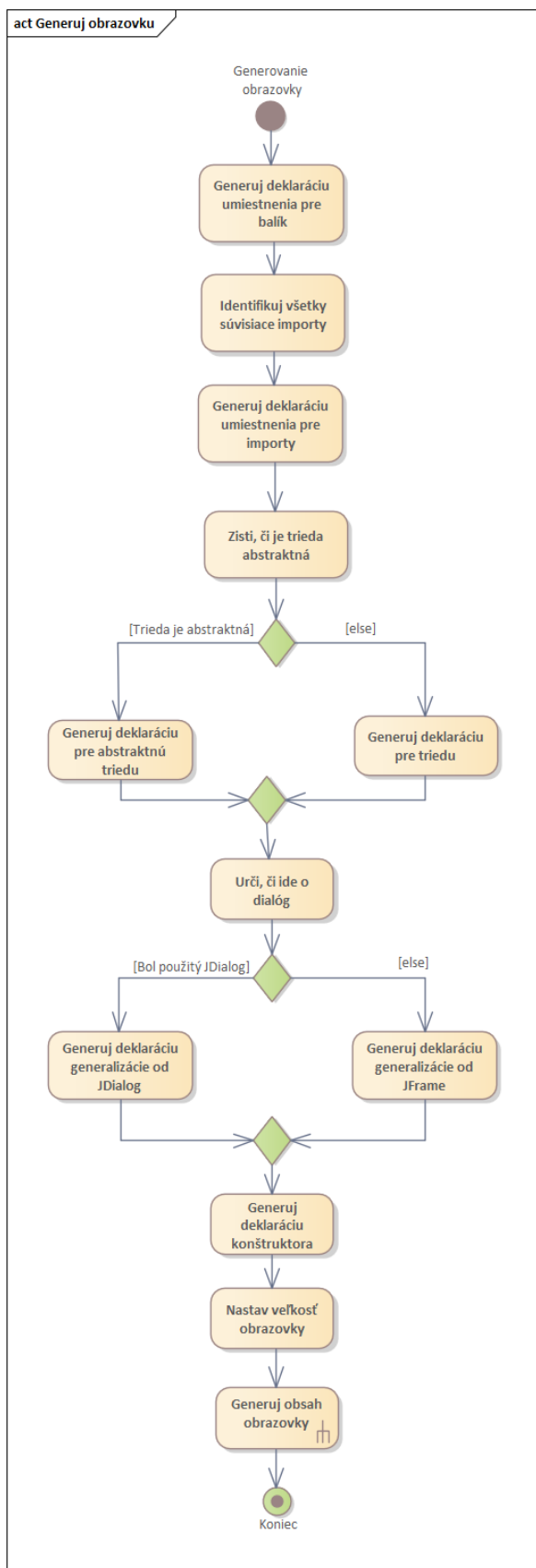
Nasledujúce diagramy aktivít (obr. 42, obr. 43, obr. 44, obr. 45, obr. 46, obr. 47, obr. 48, obr. 49) znázorňujú tok činností generovania zdrojového kódu z UML modelu:



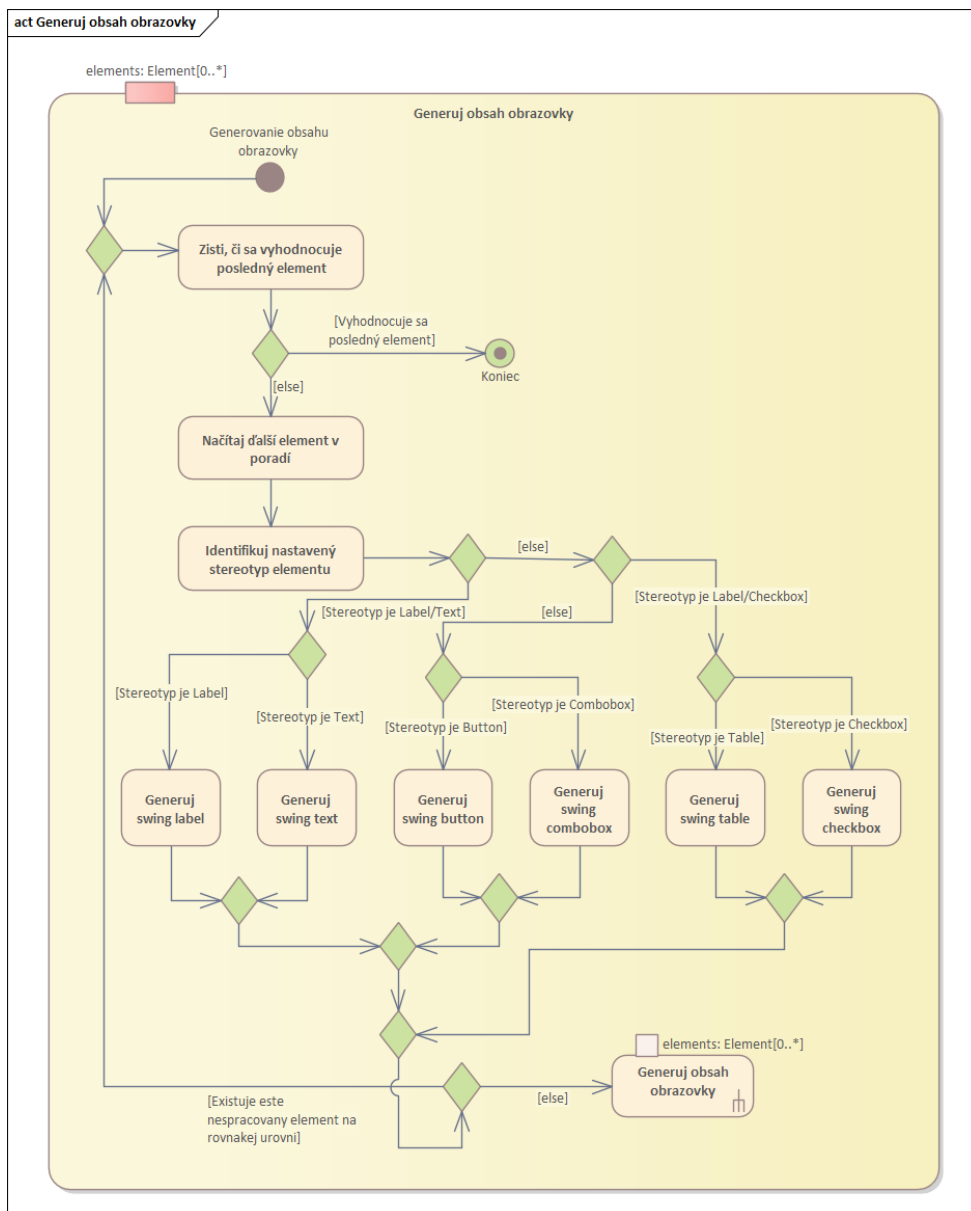
Obr. 42: Diagram aktivít - Generuj triedy swing aplikácie



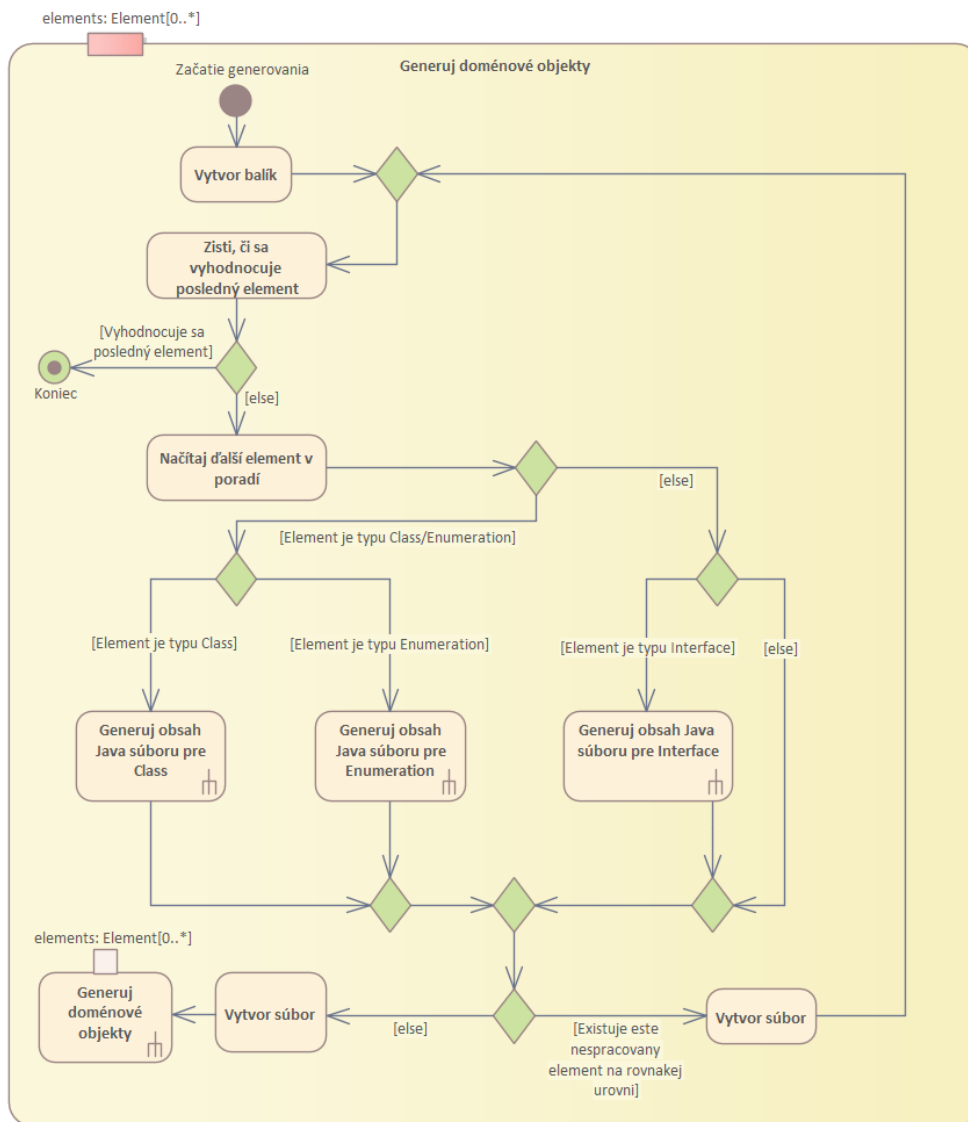
Obr. 43: Diagram aktivít - Generuj obsah Java main súboru



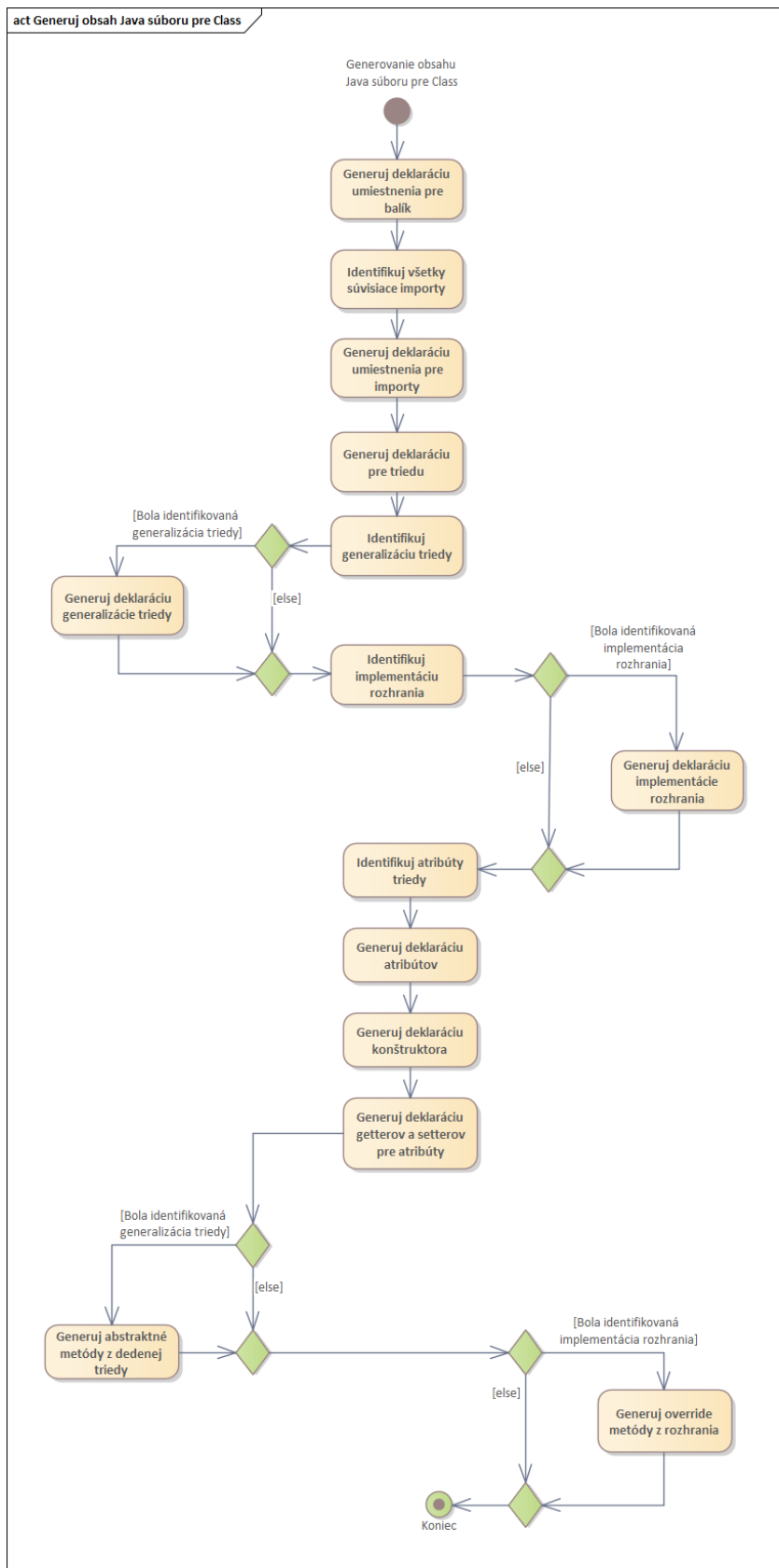
Obr. 44: Diagram aktivít - Generuj obrazovku



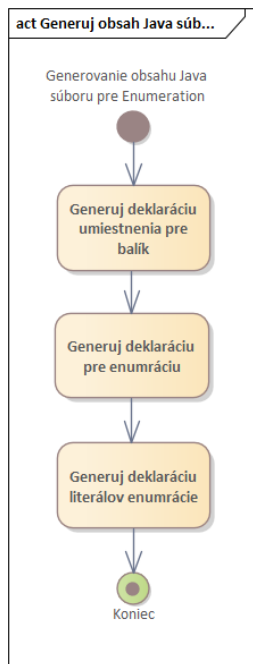
Obr. 45: Diagram aktivít - Generuj obsah obrazovky



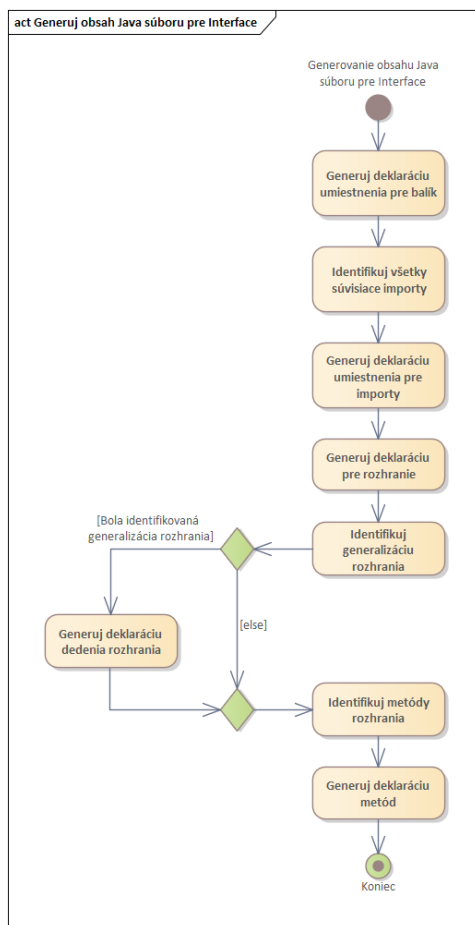
Obr. 46: Diagram aktivít - Generuj doménové objekty



Obr. 47: Diagram aktivít - Generuj obsah Java súboru pre Class



Obr. 48: Diagram aktivít - Generuj obsah Java súboru pre Enumeration



Obr. 49: Diagram aktivít - Generuj obsah Java súboru pre Interface

6. Zhrnutie a pozícia riešenia voči existujúcemu stavu poznania

Naša analýza ukázala, že problematika automatizovaného generovania používateľských rozhraní z modelov je dobre preskúmaná z hľadiska princípov modelom riadeného vývoja, no existujúce riešenia sú nejednotné podľa typov vstupných modelov, cieľových platforiem a použitých nástrojov a prostredí.

Existujú prístupy založené na UML doménových a use-case modeloch doplnených o OCL obmedzenia, ktoré umožnili generovať „form-based“ UI pre podnikové aplikácie, prístupy vychádzajúce zo scenárov a kolaboračných diagramov, ktoré generovali prototypy UI a formálne špecifikácie správania, metódy transformujúce BPMN do návrhov GUI, špecializované riešenia pre webové prototypy s LIW pre validáciu funkčných požiadaviek na použiteľnosť a pre mobilné Android aplikácie generované z anotovaných diagramov tried. UIDL jazyky ako USIXML poskytli bohatý rámec pre viacúrovňovú špecifikáciu a transformácie UI, no za cenu ďalšej jazykovej a nástrojovej vrstvy. [1] [3] [9]

Na druhej strane, integrácia GUI prototypovania priamo do UML CASE nástrojov, ako ju popísal článok „Integrating GUI Prototyping into UML Toolkit“ sa ukázala ako prakticky dôležitá, pretože umožnila zachovať väzbu medzi GUI a ostatnými architektonickými pohľadmi, podporiť modelovanie postupnosti obrazoviek a sledovanie prepojení medzi rozhraním a dátovými modelmi. Tieto riešenia sa však zameriavali predovšetkým na vizuálny prototyp a neprodukovali plnohodnotný zdrojový kód pripravený na ďalšie rozširovanie. V mnohých existujúcich prístupoch navyše vygenerovaný prototyp neodrážal presnú polohu prvkov podľa návrhu v CASE diagrame a medzi diagramom a vykonateľným UI sa nachádzala ešte jedna medzivrstva (UIDL, BPMN, anotovaný model), čo znamenalo ďalšie modelovanie a potrebnú synchronizáciu. [4] [12] [13] [14] [15] [16] [17]

Navrhované riešenie predstavuje kompromis medzi jednoduchým modelovaním GUI prototypov a komplexnými MDD rámcami, ktoré používajú vlastné jazyky a vyžadujú rozsiahlu formalizáciu. Zdieľa s nimi dôraz na modelom riadený prístup, ale odlišuje sa v niekoľkých zásadných bodoch.

Po prvé, pracuje výlučne so štandardnými prvkami UML modelu, ktoré sa v projektoch používajú: Diagramami tried a diagramami používateľského rozhrania. Nepredpokladá zavedenie

d'alšieho jazyka a tým minimalizuje potrebu učenia sa nových syntaktických a sémantických pravidiel.

Po druhé, základom riešenia je addin priamo v Enterprise Architecte, ktorý nevyžaduje export modelu do XMI ani jeho spracovanie v externých nástrojoch - namiesto toho ponúka spustenie jediným výberom z menu, po ktorom je k dispozícii vygenerovaný Java Swing kód.

Po tretie, na rozdiel od mnohých prístupov, ktoré generovali predovšetkým prototypy je našim cieľom generovať plnohodnotný zdrojový kód, ktorý možno skompilovať, spustiť a ďalej rozvíjať. Doménová vetva bude generovať triedy, rozhrania a enumerácie na základe diagramov tried, zatiaľ čo UI vetva bude generovať Swing triedy pre obrazovky a komponenty, vrátane navigačnej logiky. Vznikne tak priamy prechod od modelu k zdrojovému kódu bez potreby dodatočne ručne prepisovať prototyp do cieľového programovacieho jazyka. Navyše, obe vrstvy vzniknú z toho istého modelu, takže logická štruktúra dát a štruktúra obrazoviek zostanú konzistentné.

Po štvrté, riešenie zachováva polohu prvkov: Prvok v diagrame Enterprise Architectu má tú istú polohu ako príslušný komponent vo vygenerovanom Swing okne. To je dôležitý kvalitatívny rozdiel oproti prístupom, ktoré sa spoliehajú na abstraktné „layouty“ alebo automatické rozmiestnenie a v ktorých analytik nemá zaručené, že vizuálny prototyp bude zodpovedať navrhnutému rozloženiu prvkov. V našom prístupe analytik posunie prvok (napríklad tlačidlo) v diagrame, spustí generovanie znova a v prototypovej aplikácii vidí presne tú istú zmenu, čo výrazne uľahčuje vizuálnu komunikáciu so zákazníkom.

Po piate, správanie vygenerovaných komponentov je nastavené priamo v UML modeli pomocou hodnôt v tagoch, nie prostredníctvom d'alšieho špeciálneho jazyka.

Navrhované riešenie umožní, aby sa prototyp používal ako živý komunikačný nástroj so zákazníkom bez potreby vytvárať ďalšie, samostatné modely v iných jazykoch.

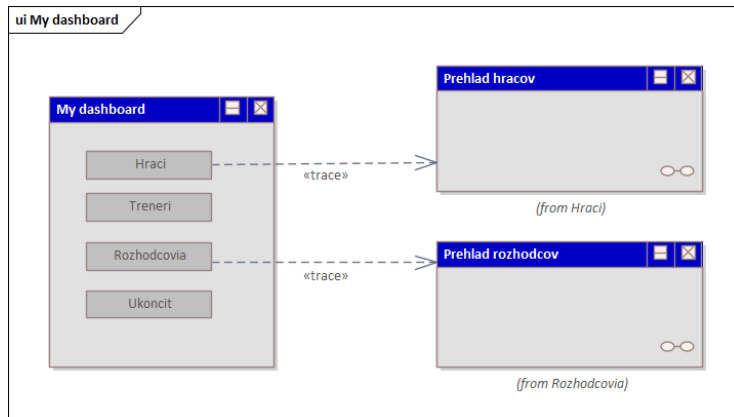
Zákazník uvidí prototyp vygenerovaný priamo z diagramov, ktoré sa v projekte bežne vytvárajú a svoje pripomienky môže dávať v podobe „posuňte toto tlačidlo“, „rozšírte toto pole“, „pridajte túto položku“. Tieto zmeny sa vykonajú v Enterprise Architecte a v novom prototypu sa následne prejavia po spustení addinu. Model v EA je pritom považovaný za východiskový zdroj, z ktorého sa dá kód kedykoľvek znovu vygenerovať, čo znižuje riziko nezhody medzi UML modelom a implementáciou. Týmto spôsobom náš prístup pridáva k existujúcim metódam

najmenej možnej dodatočnej práce a zároveň prináša benefit automatického generovania kódu, ktorý je možné použiť ako základ pre implementáciu.

7. Opis implementácie - BP1

V implementovanom addine pre Enterprise Architect sme do menu integrovali položku, ktorej aktivácia spustí generovanie prototypu do výstupného adresára a nad koreňovým balíkom identifikovaným konkrétnym GUID. Generovanie spustíme len vtedy, keď názov koreňového balíka obsahuje reťazec v okrúhlych zátvorkách. Tento reťazec následne použijeme ako názov koreňového Java balíka. Následne rekurzívne prechádzame vnorené balíky, pričom názvy balíkov normalizujeme (odstránime diakritiku, ponecháme alfanumerické znaky a medzery) a medzery mapujeme na bodky, z čoho vytvárame adresárovú štruktúru a deklarácie balíkov. V každom balíku vyhladáme elementy typu „Screen“, ku ktorým existuje diagram a pre každý takýto „Screen“ vygenerujeme Java triedu typu „JFrame“ s manuálnym rozmiestnením komponentov, s názvom okna nastaveným z „Alias“, centrovaním a veľkosťou odvodenou zo súradníc. Jednotlivé UI prvky generujeme podľa stereotypu na „JLabel“, „JTextField“, „JButton“, „JComboBox“ a „JTable“ v „JScrollPane“, pričom každému nastavíme absolútnu pozíciu a rozmery na základe prepočítaných diagramových súradníc. „JTextField“ doplníme o „ToolTip“ z „Alias“ a „JLabel“ vytvoríme s textom z „Name“. Komponent „JComboBox“ je prepojený s enumeráciou prostredníctvom konektora typu „Realization“ a pri inicializácii sa naplní položkami odvodenými z „Alias“ hodnôt jej literálov. Pri tlačidlách najprv vyhodnotíme tag „actionType“ a podľa jeho hodnoty vygenerujeme príslušnú akciu: Buď zatvorenie aktuálneho okna (tag „zavrietOkno“), ukončenie aplikácie (tag „ukoncitAplikaciju“) alebo prechod na inú obrazovku (tag „otvoritOkno“). V prípade prechodu sa otvorí obrazovka, na ktorú tlačidlo odkazuje - teda tá, ktorá je s tlačidlom prepojená konektorom so stereotypom „Trace“. Doplníme importy. Zároveň v koreňovom balíku generujeme spúšťačiu triedu „StartPrototyp“.

Ukážka na konkrétnom príklade: V EA sme namodelovali UI diagramy - na obr. 18 je prvý z nich, nastavili sme jeho tlačidlám hodnoty „Alias” (obr. 19, obr. 20, obr. 21) a tagy „actionType” (obr. 22, obr. 23, obr. 24):



Obr. 18: UI diagram obrazovky „My dashboard“

Element	Tags
General	
Name	Hraci
Type	GUIElement
Stereotype	EAUI::button
Alias	Hráči
Keywords	

Obr. 19: Nastavenie hodnoty „Alias“ tlačidla „Hraci“

Element	Tags
General	
Name	Treneri
Type	GUIElement
Stereotype	EAUI::button
Alias	Tréneri
Keywords	

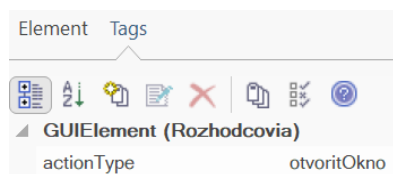
Obr. 20: Nastavenie hodnoty „Alias“ tlačidla „Treneri“

Element	Tags
General	
Name	Ukoncit
Type	GUIElement
Stereotype	EAUI::button
Alias	Ukončiť
Keywords	

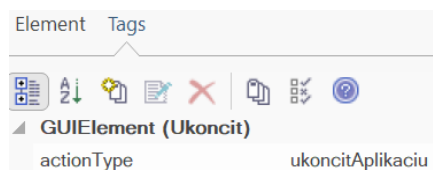
Obr. 21: Nastavenie hodnoty „Alias“ tlačidla „Ukoncit“

Element	Tags
GUIElement (Hraci)	
actionType	otvoritOkno

Obr. 22: Nastavenie tagu „actionType“ tlačidla „Hraci“



Obr. 23: Nastavenie tagu „actionType“ tlačidla „Rozhodcovia“



Obr. 24: Nastavenie tagu „actionType“ tlačidla „Ukoncit“

Príklad súradníc a rozmerov tlačidla „Hraci“ uložený v databázovom repozitári EA - obr. 25:

GUIElement:Hraci Left: 127 x Top: 93 - Width: 90 x Height: 20

Obr. 25: Súradnice a rozmery tlačidla „Hraci“

Môj addin prepočíta EA súradnice všetkých používaných prvkov do formátu používaného vo Swingu.

Po spustení nám addin vygeneroval zdrojový kód nášho prototypu aplikácie. Nasledovný kód (obr. 26) patrí obrazovke „My dashboard“:

```
package sk.fiit.swing.obrazovky.intranet.my.dashboard;

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.JButton;
import javax.swing.JComboBox;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import sk.fiit.swing.obrazovky.intranet.hraci.*;
import sk.fiit.swing.obrazovky.intranet.rozhodcovia.*;

public class MyDashboard extends JFrame {

    public MyDashboard() {

        setTitle("");

        getContentPane().setLayout(null);
        setLocationRelativeTo(null);
        setSize(136, 182);

        JButton buttonHraci = new JButton("Hráči");
        buttonHraci.setBounds(26,23,90,20);
        getContentPane().add(buttonHraci);
        buttonHraci.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                PrehľadHracov prehľadHracov = new PrehľadHracov();
                prehľadHracov.setVisible(true);
            }
        });

        JButton buttonRozhodcovia = new JButton("Rozhodcovia");
        buttonRozhodcovia.setBounds(26,88,90,20);
        getContentPane().add(buttonRozhodcovia);
        buttonRozhodcovia.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                PrehľadRozhodcov prehľadRozhodcov = new PrehľadRozhodcov();
                prehľadRozhodcov.setVisible(true);
            }
        });

        JButton buttonTreneri = new JButton("Tréneri");
        buttonTreneri.setBounds(26,53,90,20);
        getContentPane().add(buttonTreneri);
        buttonTreneri.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
            }
        });

        JButton buttonUkoncit = new JButton("Ukončiť");
        buttonUkoncit.setBounds(26,122,90,20);
        getContentPane().add(buttonUkoncit);
        buttonUkoncit.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.exit(0);
            }
        });
    }
}
```

Obr. 26: Zdrojový kód obrazovky „My dashboard“

Vygenerovaný zdrojový kód spúšťacej triedy - obr. 27:

```
package sk.fiit.swing.obrazovky;

import sk.fiit.swing.obrazovky.intranet.hraci.*;
import sk.fiit.swing.obrazovky.intranet.hraci.pridat.upravit.hraca.*;
import sk.fiit.swing.obrazovky.intranet.my.dashboard.*;
import sk.fiit.swing.obrazovky.intranet.rozhodcovia.*;
import sk.fiit.swing.obrazovky.intranet.rozhodcovia.pridat.upravit.rozhodcu.*;

import javax.swing.UIManager;
import javax.swing.JFrame;

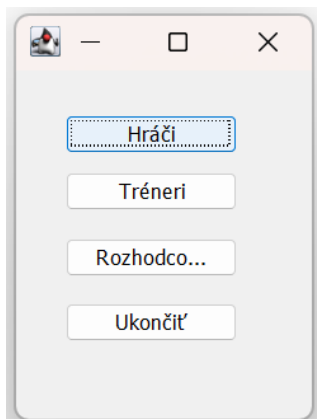
public class StartPrototyp
{
    public static void main(String[] args) {

        try
        {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }

        MyDashboard frame = new MyDashboard();
        frame.setSize(170, 220);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

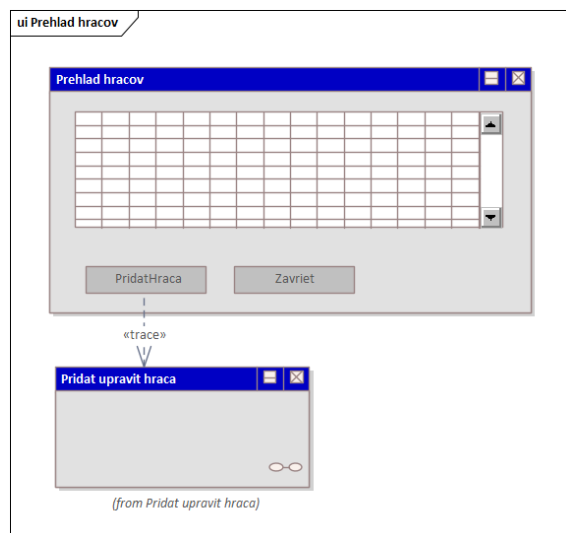
Obr. 27: Zdrojový kód spúšťacej triedy

Po spustení vygenerovaného prototypu aplikácie dostávame nasledovnú obrazovku „My dashboard“ (obr. 28), ktorú sme namodelovali v EA (obr. 18):



Obr. 28: Obrazovka „My dashboard“ v spustenom vygenerovanom prototyp aplikácie

Tlačidlo „Hraci“ je prepojené s obrazovkou „Prehľad hracov“ pomocou konektora so stereotypom „Trace“ (obr. 18) a má hodnotu tagu „actionType“ nastavenú na „otvoritOkno“ (obr. 22). Kliknutím na toto tlačidlo sa otvorí obrazovka „Prehľad hracov“, ktorej patrí nasledujúci UI diagram (obr. 29). Jeho tlačidlám sme opäť nastavili hodnoty „Alias“ (obr. 30) a tagy „actionType“:



Obr. 29: UI diagram obrazovky „Prehľad hracov“

Element	Tags
General	
Name	PridatHraca
Type	GUIElement
Stereotype	EAUI::button
Alias	Pridať hráča
Keywords	

Obr. 30: Nastavenie hodnoty „Alias“ tlačidla „PridatHraca“

Vygenerovaný zdrojový kód, ktorý patrí obrazovke „Prehľad hracov“ - obr. 31:

```
package sk.fiit.swing.obrazovky.intranet.hraci;

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.JButton;
import javax.swing.JComboBox;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import sk.fiit.swing.obrazovky.intranet.hraci.pridat.upravit.hraca.*;

public class PrehľadHracov extends JFrame {

    public PrehľadHracov() {

        setTitle("");

        getContentPane().setLayout(null);
        setLocationRelativeTo(null);
        setSize(358, 192);

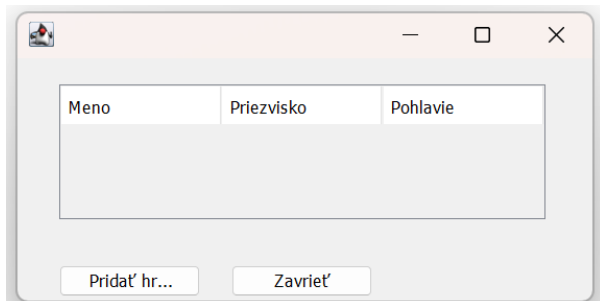
        JButton buttonPridatHraca = new JButton("Pridať hráča");
        buttonPridatHraca.setBounds(27,132,90,20);
        getContentPane().add(buttonPridatHraca);
        buttonPridatHraca.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                PridatUpravitHraca pridatUpravitHraca = new PridatUpravitHraca();
                pridatUpravitHraca.setVisible(true);
            }
        });

        String[] columnNames = {"Meno", "Priezvisko", "Pohlavie"};
        Object[][] data = {};
        JTable tableTable1 = new JTable(data, columnNames);
        JScrollPane scrollPaneTable1 = new JScrollPane(tableTable1);
        scrollPaneTable1.setBounds(27,17,311,86);
        getContentPane().add(scrollPaneTable1);

        JButton buttonZavriet = new JButton("Zavrieť");
        buttonZavriet.setBounds(137,132,90,20);
        getContentPane().add(buttonZavriet);
        buttonZavriet.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                dispose();
            }
        });
    }
}
```

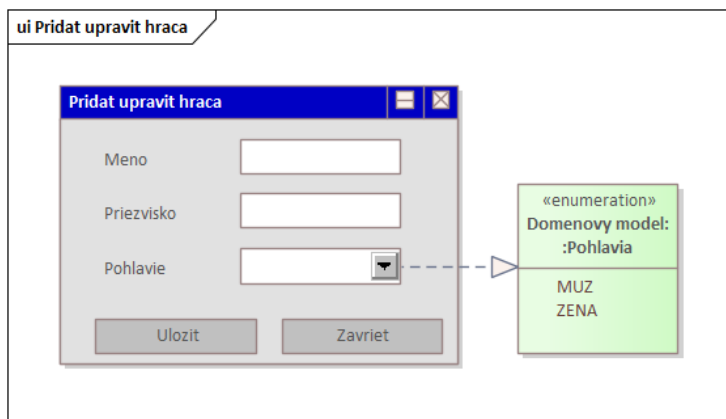
Obr. 31: Zdrojový kód obrazovky „Prehľad hracov“

Po kliknutí na tlačidlo „Hráči“ v spustenom vygenerovanom prototype aplikácie (obr. 28) dostávame obrazovku „Prehľad hracov“ (obr. 32), ktorú sme namodelovali v EA (obr. 29):



Obr. 32: Obrazovka „Prehľad hracov“ v spustenom vygenerovanom prototype aplikácie

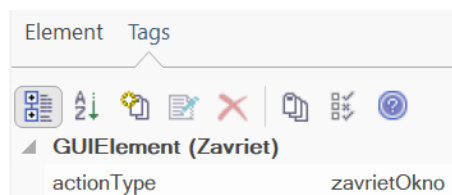
Tlačidlo „PridaťHraca“ je prepojené s obrazovkou „Pridať upraviť hraca“ pomocou konektora so stereotypom „Trace“ (obr. 29) a má hodnotu tagu „actionType“ nastavenú na „otvoritOkno“. Kliknutím na toto tlačidlo sa otvorí obrazovka „Pridať upraviť hraca“, ktorej patrí nasledujúci UI diagram (obr. 33). Jeho tlačidlám sme opäť nastavili hodnoty „Alias“ (obr. 34), tagy „actionType“ (obr. 35) a „combobox“ sme prepojili s enumeráciou prostredníctvom konektora typu „Realization“ (obr. 33). Nastavili sme hodnoty „Alias“ textovým poliam (obr. 36) aj literálom enumerácie (obr. 37):



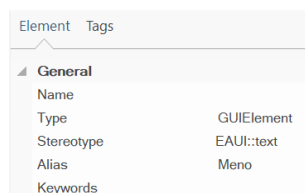
Obr. 33: UI diagram obrazovky „Pridať upraviť hraca“

Element	Tags
General	
Name	Zavrieť
Type	GUIElement
Stereotype	EAUI::button
Alias	Zavrieť
Keywords	

Obr. 34: Nastavenie hodnoty „Alias“ tlačidla „Zavrieť“



Obr. 35: Nastavenie tagu „actionType“ tlačidlu „Zavriet“



Obr. 36: Nastavenie hodnoty „Alias“ textovému polu

Attributes Operations Receptions Parts / Properties Interaction Points					
Name	Type	Scope	Stereotype	Alias	Initial Value
ZENÁ		Public	enum	Žena	
New Attribute...					
MUZ		Public	enum	Muž	

Obr. 37: Nastavenie hodnôt „Alias“ literálom enumerácie „Pohlavia“

Vygenerovaný zdrojový kód, ktorý patrí obrazovke „Pridat upravit hraca“ - obr. 38:

```
package
sk.fiit.swing.obrazovky.intranet.hraci.pridat.upravit.hraca;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.JButton;
import javax.swing.JComboBox;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JScrollPane;
import javax.swing.JTable;

public class PridatUpravitHraca extends JFrame {

    public PridatUpravitHraca() {

        setTitle("");

        getContentPane().setLayout(null);
        setLocationRelativeTo(null);
        setSize(234, 174);

        JLabel lblNewLabelMeno = new JLabel("Meno");
        lblNewLabelMeno.setBounds(20,14,90,20);
        getContentPane().add(lblNewLabelMeno);

        JTextField textFieldmeno = new JTextField();
        textFieldmeno.setBounds(101,14,90,20);
        textFieldmeno.setToolTipText("Meno");
        getContentPane().add(textFieldmeno);

        JComboBox comboBoxpohlavie = new JComboBox();
        comboBoxpohlavie.setBounds(101,75,90,20);
        comboBoxpohlavie.addItem(" ");
        comboBoxpohlavie.addItem("Muž");
        comboBoxpohlavie.addItem("Žena");
        getContentPane().add(comboBoxpohlavie);

        JLabel lblNewLabelPohlavie = new JLabel("Pohlavie");
        lblNewLabelPohlavie.setBounds(20,75,90,20);
        getContentPane().add(lblNewLabelPohlavie);

        JLabel lblNewLabelPriezvisko = new JLabel("Priezvisko");
        lblNewLabelPriezvisko.setBounds(20,44,90,20);
        getContentPane().add(lblNewLabelPriezvisko);

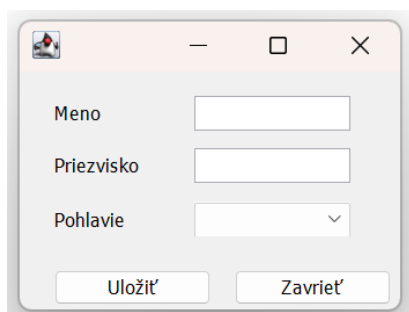
        JTextField textFieldpriezvisko = new JTextField();
        textFieldpriezvisko.setBounds(101,44,90,20);
        textFieldpriezvisko.setToolTipText("priezvisko");
        getContentPane().add(textFieldpriezvisko);

        JButton buttonUlozit = new JButton("Uložiť");
        buttonUlozit.setBounds(20,114,90,20);
        getContentPane().add(buttonUlozit);
        buttonUlozit.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                dispose();
            }
        });

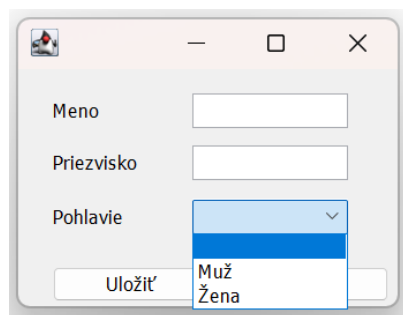
        JButton buttonZavriet = new JButton("Zavrieť");
        buttonZavriet.setBounds(124,114,90,20);
        getContentPane().add(buttonZavriet);
        buttonZavriet.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                dispose();
            }
        });
    }
}
```

Obr. 38: Zdrojový kód obrazovky „Pridat upravit hraca“

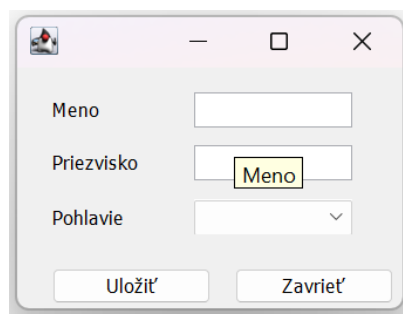
Po kliknutí na tlačidlo „Pridať hráča“ v spustenom vygenerovanom prototype aplikácie (obr. 32) dostávame obrazovku „Pridať upraviť hraca“ (obr. 39, obr. 40, obr. 41), ktorú sme namodelovali v EA (obr. 33):



Obr. 39: Obrazovka „Pridať upraviť hraca“ v spustenom vygenerovanom prototype aplikácie



Obr. 40: Obrazovka „Pridať upraviť hraca“ v spustenom vygenerovanom prototype aplikácie s rozbaleným „comboboxom“



Obr. 41: Obrazovka „Pridať upraviť hraca“ v spustenom vygenerovanom prototype aplikácie so zobrazenou nápovedou textového pola

Postup spustenia addinu

Pre spustenie addinu v 64-bitovej verzii EA je potrebné pracovať s riešením vo Visual Studiu a mať v prostredí dostupný .NET Framework 4.7.2. V riešení sú dva projekty a je potrebné, aby projekt NikolaSasinkovaBP1 mal v referenciách pridaný projekt NikolaSasinkovaBP1GenerateSwing. V oboch projektoch je potrebné pridať referenciu na Interop.EA.dll a System.Windows.Forms. V oboch projektoch je potrebné nastaviť Target Framework na .NET Framework 4.7.2 (Properties -> Application -> Target framework) a nastaviť platformu na x64 (Properties -> Build -> Platform target). Je potrebné zapnúť COM viditeľnosť (Properties -> Application -> Assembly Information -> „Make assembly COM-Visible“) a zapnúť „Register for COM interop“ (Properties -> Build). Aby Enterprise Architect addin načítal, je potrebné pridať nový kľúč s názvom „namespace“ do registra operačného systému Windows do vetvy HKEY_CURRENT_USER\Software\Sparx Systems\EAAddins64. Jeho hodnota sa musí nastaviť na „NikolaSasinkovaBP1GenerateSwing.NikolaSasinkovaBP1GenerateSwingClass“ pri „NikolaSasinkovaBP1GenerateSwing“ a „NikolaSasinkovaBP1.NikolaSasinkovaBP1Class“ pri „NikolaSasinkovaBP1“. Nakoniec je potrebné riešenie skompilovať (pomocou „Build“) a následne spustiť Enterprise Architect.

Zhodnotenie aktuálneho stavu riešenia

V tejto fáze je zrejmé, že zvolený prístup má zmysel a je prakticky realizovateľný: Z modelu vytvoreného v Enterprise Architecte je možné vygenerovať spustiteľný prototyp používateľského rozhrania, ktorý je zrozumiteľný pre zákazníka a umožňuje včasné overenie návrhu bez zdĺhavého ručného prototypovania. Doterajší stav práce ukazuje, že takýto postup môže výrazne skrátiť spätnú väzbu, znížiť chybovosť spôsobenú nepochopením formálnych diagramov a podporiť konzistenciu medzi návrhom a výsledným výstupom, keďže prototyp vzniká priamo z modelu. Zároveň však ide o rozpracované riešenie - aktuálna implementácia pokrýva predovšetkým jadro potrebné na praktické overenie navrhnutého prístupu.

Plán ďalšieho rozvoja

Ďalšie pokračovanie práce vychádza z navrhnutého riešenia, podľa ktorého má addin v Enterprise Architecte po spustení vygenerovať zdrojový kód prototypu aplikácie rozdelený na doménovú a používateľskú časť, pričom doménová časť sa bude odvodzovať z diagramov tried a bude zahŕňať triedy, rozhrania a enumerácie a používateľská časť sa doplní tak, aby pokrývala plánovaný rozsah typov obrazoviek a prvkov a aby sa zachovalo konzistentné prepojenie medzi rozhraním a doménou. Výsledkom má byť spustiteľný zdrojový kód, ktorý zodpovedá štruktúre modelu a zároveň podporuje prístup, v ktorom je model v EA hlavný zdroj informácií a kód sa z neho dá opakovane generovať.

Zdroje

- [1]<https://ieeexplore.ieee.org/document/4335249>, 04.10.2025.
- [2]<https://ieeexplore.ieee.org/document/1409919>, 04.10.2025.
- [3]<https://ieeexplore.ieee.org/document/777995>, 04.10.2025.
- [4]https://www.researchgate.net/publication/265803265_Integrating_GUI_Prototyping_into_UML_Toolkit, 07.10.2025.
- [5]https://www.researchgate.net/publication/250762832_USIXML_a_User_Interface_Description_Language_for_Specifying_Multimodal_User_Interfaces, 07.10.2025.
- [6]https://www.researchgate.net/publication/216394137_FlexiXML_-_A_portable_user_interface_rendering_engine_for_UsiXML, 07.10.2025.
- [7]<https://ieeexplore.ieee.org/document/7336026>, 07.10.2025.
- [8]<https://ieeexplore.ieee.org/document/7018489>, 10.10.2025.
- [9]<https://ieeexplore.ieee.org/document/10063755>, 10.10.2025.
- [10]<https://ieeexplore.ieee.org/document/919177>, 10.10.2025.
- [11]https://sparxsystems.com/enterprise_architect_user_guide/17.1/add-ins_scripting/reference.html, 18.10.2025.
- [12]<https://www.pst.ifi.lmu.de/~kochn/pUML2001-Hen-Koch.pdf>, 20.10.2025.
- [13]https://www.researchgate.net/publication/221320216_A_UML_profile_for_GUI_layout, 03.11.2025.
- [14]https://www.researchgate.net/publication/274494235_Languages_for_model-driven_development_of_user_interfaces_Review_of_the_state_of_the_art, 05.11.2025.
- [15]<https://ieeexplore.ieee.org/document/8406675>, 05.11.2025.
- [16]https://nch.health.gov.ng/wp-content/uploads/2025/10/Usixml_A_User_Interface_Description_Lang-2.pdf, 05.11.2025.
- [17]<https://ceur-ws.org/Vol-159/paper6.pdf>, 11.11.2025.
- [18]<https://andromda.sourceforge.net/>, 16.11.2025.

- [19]<https://marketplace.eclipse.org/content/papyrus-moka>, 16.11.2025.
- [20]<https://eclipse.dev/papyrus/>, 18.11.2025.
- [21]<https://www.integranova.com/en/integranova-m-e-s/blog/2021/03/model-driven-code-generation/>, 18.11.2025.
- [22]<https://www.integranova.com/en/integranova-m-e-s/blog/2021/05/use-cases-integranova/>, 20.11.2025.
- [23]<https://www.3ds.com/products/catia/no-magic/cameo-simulation-toolkit>, 20.11.2025.
- [24]<https://docs.nomagic.com/spaces/CST190SP3/pages/47107393/Cameo+Simulation+Toolkit+Documentation>, 20.11.2025.
- [25]<https://www.3ds.com/products/catia/no-magic/magicdraw>, 21.11.2025.
- [26]<https://www.webratio.com/site/content/en/home>, 22.11.2025.
- [27]<https://www.ifml.org/>, 23.11.2025.
- [28]<https://www.visual-paradigm.com/>, 23.11.2025.
- [29]<https://eclipse.dev/emf/>, 23.11.2025.
- [30]<https://projects.eclipse.org/projects/modeling.emf>, 24.11.2025.
- [31]<https://eclipsesource.com/blogs/tutorials/emf-tutorial/>, 24.11.2025.
- [32]<https://eclipse.dev/acceleo/>, 25.11.2025.
- [33]<https://projects.eclipse.org/projects/modeling.acceleo>, 25.11.2025.
- [34]https://wiki.eclipse.org/Acceleo/Getting_Started, 25.11.2025.
- [35]<https://pyuibuilder.com/>, 28.11.2025.
- [36]<https://www.animaapp.com/>, 28.11.2025.
- [37]<https://www.figma.com/community/plugin/857346721138427857/anima-figma-to-code-react-html-css-tailwind-mui-devmode-inspect-react-html-vue-css>, 29.11.2025.
- [38]<https://www.builder.io/>, 29.11.2025.

- [39]<https://www.figma.com/community/plugin/747985167520967365/builder-io-figma-to-code-ai-apps-react-vue-tailwind-etc>, 29.11.2025.
- [40]<https://teleporthq.io/>, 30.11.2025.
- [41]<https://www.figma.com/community/plugin/992726161890204477/teleporthq-figma-to-code-export-html-css-react-vue>, 30.11.2025.
- [42]<https://www.formdev.com/>, 30.11.2025.
- [43]<https://marketplace.eclipse.org/content/jformdesigner-swing-gui-designer>, 30.11.2025.
- [44]<https://eclipse.dev/windowbuilder/>, 01.12.2025.
- [45]<https://marketplace.eclipse.org/content/windowbuilder#details>, 01.12.2025.
- [46]<https://netbeans.apache.org/tutorial/main/kb/docs/java/quickstart-gui/>, 01.12.2025.